

Formal Verification Report: Armada Crowdfund & Governance

-
- **Date:** May, 2026
 - **Audit Repo:** [Cyfrin/audit-2026-04-armada-crowdfund-governance](#).
 - **Client Repo:** [ship-armada/armada-poc](#).
 - **Audit Commit:** a009580c1f6ad6fc1aa979a92687827194fa8b68
 - **Mitigation Commit:** 5b525ef8c9a7225d60e35b5b08b47dad1963c94f
 - **Author:** [Arion\(@cyfrin\)](#) private formal verification engagement)
 - **Certora Prover version:** 8.8.1

Table of Contents

1. [About Armada](#)
2. [Formal Verification Methodology](#)
3. [Project Structure and Scope](#)
4. [Verification Assumptions](#)
5. [Verification Properties](#)
 - [Crowdfund](#)
 - [Revenue](#)
 - [Winddown](#)
 - [Treasury-Gov / Steward](#)
 - [ArmadaToken / Governance Support](#)
 - [ArmadaGovernor](#)
6. [Findings and Real Issues](#)
7. [Setup and Execution](#)
8. [Resources](#)

About Armada

Armada is a governance and treasury protocol built around the ARM token, decentralized proposal execution, controlled treasury spending, revenue tracking, and an orderly wind-down/redemption path. The verified scope focuses on the core lifecycle of the protocol: ARM distribution, token voting power, governance proposal execution, treasury controls, emergency controls, revenue-based locks, and post-wind-down redemption.

The crowdfund system verifies the ARM sale and allocation lifecycle, including whitelist participation, invite-hop accounting, USDC commitment, capped demand, finalization, ARM claims, and refunds. The ARM token verification covers ERC20 behavior, vote/delegation behavior, transfer restrictions before wind-down, and delegation restrictions for locked treasury-like addresses.

Governance verification covers the Armada Governor, typed proposal flows, timelock scheduling/execution, quorum/timing rules, security council actions, steward proposals, and wind-down behavior. Treasury verification covers ArmadaTreasuryGov and TreasurySteward, including direct treasury distributions, steward budget usage, aggregate outflow limits, delayed activation for looser limits, immediate tightening, and steward lifecycle rules.

Formal Verification Methodology

Formal verification was used to reason about the Armada contracts against explicitly written correctness properties. Rather than exercising a selected set of examples, as in unit tests or fuzz tests, the Certora Prover analyzes the compiled contracts symbolically and checks whether the specified properties hold for all relevant inputs, callers, contract states, and execution paths within the verification model.

The specifications were written in CVL, the Certora Verification Language. Each CVL rule captures an expected protocol guarantee, such as preserving accounting invariants, enforcing access control, respecting lifecycle transitions, or rejecting invalid actions.

The Solidity contracts, harnesses, and CVL specifications are then submitted to the prover, which translates them into logical constraints and attempts to either prove the property or produce a counterexample showing how it can be violated. This methodology complements testing by covering broad state spaces that are difficult to enumerate manually. Tests are useful for concrete scenarios and integration behavior, while formal verification is used here to check general safety properties across the protocol's critical flows.

Types of Properties

Valid State: invariants and global correctness conditions that should hold across reachable states.

Variable Transition: rules proving that selected state variables change only in expected ways.

State Transition: authorization, gating, and lifecycle transition properties.

High-Level: end-to-end economic and business-logic properties.

Unit / Liveness: reachability and model sanity checks.

Negative / Anti-Property: offensive witnesses or expected-failure checks documenting what must not be assumed.

Project Structure and Scope

The Certora workspace is organized into run configurations, Solidity harnesses, execution scripts, CVL specifications, and the consolidated properties catalogue.

```
certora/ # Certora verification workspace
├── configs/ # Certora run configurations
│   ├── Crowdfund/ # Crowdfund verification configs
│   │   ├── CFAccounting.conf # Runs crowdfund accounting properties
│   │   ├── CFAllocation.conf # Runs allocation properties
│   │   ├── CFCancelInvariants.conf # Runs cancellation invariants
│   │   ├── CFClaimSettlement.conf # Runs claim settlement properties
│   │   ├── CFCommitment.conf # Runs commitment properties
│   │   ├── CFElasticWaterfall.conf # Runs elastic waterfall properties
│   │   ├── CFFinalization.conf # Runs finalization properties
│   │   ├── CFInviteGraph.conf # Runs invite graph properties
│   │   ├── CFRoundingPrecision.conf # Runs rounding precision properties
│   │   ├── CFSignatureNonce.conf # Runs signature nonce properties
│   │   ├── CFStateMachine.conf # Runs state machine properties
│   │   └── CFTimingInvariants.conf # Runs timing invariants
│   ├── Governance/ # Governance verification configs
│   │   ├── AdapterRegistry.conf # Runs adapter registry properties
│   │   ├── ArmadaGovernorAdmin.conf # Runs governor admin properties
│   │   ├── ArmadaGovernorBookkeeping.conf # Runs proposal bookkeeping properties
│   │   ├── ArmadaGovernorClassification.conf # Runs governor classification properties
│   │   ├── ArmadaGovernorClassificationPart2.conf # Runs extended classification properties
│   │   ├── ArmadaGovernorComplement.conf # Runs combined governor complement suite
│   │   ├── ArmadaGovernorLifecycle.conf # Runs lifecycle properties
│   │   ├── ArmadaGovernorLifecyclePart2.conf # Runs extended lifecycle properties
│   │   ├── ArmadaGovernorProposals.conf # Runs proposal creation properties
│   │   ├── ArmadaGovernorState.conf # Runs governor state invariants
│   │   ├── ArmadaGovernorVeto.conf # Runs veto properties
│   │   ├── ArmadaGovernorVetoPart2.conf # Runs extended veto properties
│   │   ├── ArmadaGovernorVotes.conf # Runs concrete vote-path properties
│   │   ├── ArmadaGovernorVotesPart2.conf # Runs vote accounting properties
│   │   ├── ArmadaToken_extended.conf # Runs extended token properties
│   │   ├── RevenueCounter.conf # Runs revenue counter properties
│   │   ├── RevenueLock.conf # Runs revenue lock properties
│   │   ├── ShieldPauseController.conf # Runs shield pause controller properties
│   │   ├── TreasuryGov.conf # Runs treasury governance properties
│   │   ├── TreasuryGovValueTransfer.conf # Runs concrete ERC20 value-transfer properties
│   │   ├── TreasurySteward.conf # Runs treasury steward properties
│   │   ├── XLockedNoVotes.conf # Runs locked-address voting exclusion properties
│   │   └── XRateLimit.conf # Runs cross-contract rate-limit properties
│   ├── WindDown/ # Wind-down verification configs
│   │   ├── ArmadaRedemption_extended.conf # Runs redemption properties
│   │   ├── ArmadaWindDown_extended.conf # Runs wind-down properties
│   │   └── XWindDownAtomic.conf # Runs atomic wind-down properties
├── harness/ # Solidity harness contracts
│   ├── ArmadaTreasuryGovHarness.sol # Harness for treasury governance internals and helpers
│   ├── ArmadaTreasuryGovValueTransferHarness.sol # Harness for concrete value-transfer checks
│   ├── CrowdFundHarness.sol # Harness for crowdfund verification
│   └── GovernanceHarness.sol # Harness for ArmadaGovernor verification
└── scripts/ # Helper scripts
```

```

└─ certora_run_all.sh # Runs Certora configs in batch
└─ specs/ # CVL specifications
└─ ArmadaToken/ # ArmadaToken specs
└─ ArmadaToken.spec # ArmadaToken core properties
└─ ERC20.spec # ERC20 behavior properties
└─ ERC20Wrapper.spec # ERC20 wrapper properties
└─ Helpers.spec # Shared token helper definitions
└─ IERC20.spec # IERC20 interface properties
└─ IERC2612.spec # Permit interface properties
└─ Crowdfund/ # Crowdfund specs
└─ CFAccounting.spec # Accounting properties
└─ CFAllocation.spec # Allocation properties
└─ CFCancelInvariants.spec # Cancellation invariants
└─ CFClaimSettlement.spec # Claim settlement properties
└─ CFCommitment.spec # Commitment properties
└─ CFElasticWaterfall.spec # Elastic waterfall properties
└─ CFFinalization.spec # Finalization properties
└─ CFInviteGraph.spec # Invite graph properties
└─ CFRoundingPrecision.spec # Rounding precision properties
└─ CFSignatureNonce.spec # Signature nonce properties
└─ CFStateMachine.spec # State machine properties
└─ CFTimingInvariants.spec # Timing invariants
└─ Governance/ # Governance rule specs
└─ ArmadaGovernor/ # ArmadaGovernor rule specs
└─ ArmadaGovernor_admin.spec # Admin, bootstrap, and constant properties
└─ ArmadaGovernor_bookkeeping.spec # Proposal metadata and bookkeeping properties
└─ ArmadaGovernor_classification.spec # Selector and call classification properties
└─ ArmadaGovernor_lifecycle.spec # Proposal lifecycle properties
└─ ArmadaGovernor_part2_classification.spec # Extended classification checks
└─ ArmadaGovernor_part2_lifecycle.spec # Extended lifecycle and timing checks
└─ ArmadaGovernor_part2_veto.spec # Extended veto and ratification checks
└─ ArmadaGovernor_part2_votes.spec # Vote tally, quorum, and threshold checks
└─ ArmadaGovernor_proposals.spec # Proposal typing and outflow checks
└─ ArmadaGovernor_state.spec # Governor storage invariants
└─ ArmadaGovernor_veto.spec # Security Council veto properties
└─ ArmadaGovernor_votes.spec # Concrete vote transition properties
└─ GovernanceComplement.spec # Compatibility wrapper for focused governor suites
└─ AdapterRegistry.spec # Adapter authorization properties
└─ GovernanceConstants.spec # Governance constant definitions
└─ ShieldPauseController.spec # Shield pause and wind-down authorization properties
└─ TreasuryGov.spec # Treasury governance properties
└─ TreasuryGovValueTransfer.spec # Concrete ERC20 transfer delta properties
└─ TreasurySteward.spec # Steward identity and term properties
└─ XLockedNoVotes.spec # Locked-address vote exclusion properties
└─ XRateLimit.spec # Cross-contract outflow rate-limit properties
└─ Revenue/ # Revenue specs
└─ RevenueCounter.spec # Revenue counter properties
└─ RevenueLock.spec # Revenue lock properties
└─ setup/ # Shared setup specs
└─ ERC20/ # Shared ERC20 CVL models and summaries
└─ erc20cvl.spec # ERC20 ghost state and transfer summaries
└─ erc20dispatched.spec # ERC20 dispatch model
└─ erc20votes3.spec # ERC20Votes checkpoint summaries
└─ Governance/ # Shared governance setup specs
└─ ArmadaGovernor/ # ArmadaGovernor setup specs
└─ ArmadaGovernorConcreteProposalSetup.spec # Concrete proposal setup with hooks
└─ ArmadaGovernorConcreteProposalSetup_nohooks.spec # Concrete proposal setup without hooks
└─ ArmadaGovernor_part2_setup.spec # Shared part2 methods, ghosts, hooks, and helpers
└─ ArmadaGovernor_part2_setup_noghosts.spec # Admin-suite setup without ghost hooks
└─ GovernanceComplementSetup.spec # Shared complement-suite setup and hooks
└─ GovernanceSetup.spec # Foundational governor setup
└─ TreasuryGovSetup.spec # Shared TreasuryGov setup
└─ TreasuryGovValueTransferSetup.spec # Shared value-transfer setup
└─ TreasuryStewardSetup.spec # Shared TreasurySteward setup
└─ Constants.spec # Shared CVL constants
└─ CrowdFundSetup.spec # Shared crowdfund setup
└─ CrowdFundSetup_summarized.spec # Summarized crowdfund setup
└─ WindDown/ # Wind-down specs
└─ ArmadaRedemption.spec # Redemption properties
└─ ArmadaWindDown.spec # Wind-down properties
└─ XWindDownAtomic.spec # Atomic wind-down properties
└─ properties.md # Property catalog and documentation

```

Verification Assumptions

The verification should be read under the assumptions below. Some assumptions are safe proof dependencies, while others are explicit modeling limitations introduced to keep the prover tractable.

Safe Assumptions

- ERC20 and ARM token accounting invariants are reused as proof dependencies, including **totalSupplyIsSumOfBalances**, **zeroAddressNoBalance**, **noAllowance**, and **totalSupplyIsSmallerThanUnderlyingBalance**.
- Crowdfund state and accounting invariants are reused for dependent rules. These include active/finalized state consistency, finalized capped-demand correctness per hop, allocated USDC/ARM consistency...etc.
- ShieldPauseController invariants are reused for wind-down pause state, including the relationship between the stored wind-down controller address and the post-wind-down pause counter.
- TreasuryGov invariants are reused for pending rate-limit configurations, including pending BPS range, pending window minimum, and pending absolute limit floor constraints.
- Harness assumptions expose internal state or bound aggregate checks without changing rule intent. They are used to make private/internal protocol state observable to the prover and to express accounting properties cleanly.

Unsafe Assumptions / Prover Limitations

- Most specs use bounded loop unrolling with `loop_iter = 3`. meaning paths requiring more loop iterations than the configured bound may be excluded.
- Crowdfund participant aggregation is bounded through `CrowdFundHarness.MAX_PARTICIPANTS = 10`. Properties over participant-node arrays and capped demand are therefore verified under the bounded harness model.
- Crowdfund capped-demand logic is summarized in some specs, especially around `_iterateCappedDemand`, using ghost state and bounded helper functions. To avoid looping on all the participants' nodes.
- ERC20 behavior is abstracted in many specs through `erc20cvl.spec`. This proves protocol logic against a well-formed ERC20 model, while full ARM token behavior is covered separately in **ArmadaToken** specs.
- ERC20Votes behavior is abstracted in Governor specs through vote/delegate ghosts.
- Several Governor and Timelock interactions are summarized as **NONDET** or **NONDET DELETE**, including timelock hashing, scheduling, execution, cancellation, and selected internal guard checks in focused specs. This isolates Governor behavior from Timelock internals and should be treated as modular verification.

Verification Properties

ArmadaCrowdfund.sol

Commitment Accounting (19 properties)

Specification: [specs/Crowdfund/CFCommitment.spec](#)

Rules passed: 19 Passed | Certora run: [CFCommitment.conf](#)

Mitigation run: [CFCommitment.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-2	VT_commitExactDeltas	A successful <code>commit(h, amount)</code> increments <code>committed[msg.sender][h]</code> , <code>hopStats[h].totalCommitted</code> , and <code>totalCommitted</code> by exactly amount	✓	✓	
CF-3	VT_commitUsdcTransferMatchesAmount	A successful <code>commit(h, amount)</code> decreases sender USDC and increases crowdfund USDC by exactly amount	✓	✓	
CF-4	VT_commitWithInviteExactDeltas	A successful <code>commitWithInvite(inviter, fromHop, nonce, deadline, sig, amount)</code> applies the same exact commitment	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
		and USDC deltas at inviteeHop = fromHop + 1			
CF-5	VT_committedMonotonicPerParticipant	committed[addr][h] is monotonic non-decreasing for every participant and hop	✓	✓	
CF-6	VT_totalCommittedMonotonic	totalCommitted is monotonic non-decreasing	✓	✓	
CF-7	VS_commitRequiresWhitelist	commit(h, _) reverts unless the caller is whitelisted at hop h	✓	✓	
CF-8	VS_commitRevertsBelowMinimum	commit and commitWithInvite reject amount < MIN_COMMIT	✓	✓	
CF-9	VS_overCapDepositsPermitted	Over-cap deposits remain permitted; caps affect allocation/refund, not the ability to commit	✓	✓	
CF-10	VS_launchTeamCannotCommit	launchTeam cannot commit USDC	✓	✓	
CF-11	ST_commitWithInviteNonceConsumedOnlyOnSuccess	commitWithInvite consumes the inviter nonce only if the call succeeds	✓	✓	
CF-12	ST_commitWithInviteRevertsAfterPhaseChange	commitWithInvite reverts after the sale leaves Active	✓	✓	
CF-13	HL_singleNodeCappedDemandConsistency	In a single-node finalized sale, stored hopStats[h].cappedCommitted and cappedDemand equal that node's capped demand	✓	✓	
CF-14	HL_totalCommittedIsSumOfCommitted	totalCommitted == $\sum_{addr,h}$ committed[addr][h]	✓	✓	
CF-15	HL_totalCommittedEqualsHopSum	totalCommitted == hopTotalCommitted[0] + hopTotalCommitted[1] + hopTotalCommitted[2]	✓	✓	
CF-16	HL_hop0TotalCommittedMatchesCommittedSum	For each hop h, hopStats[h].totalCommitted == $\sum_{addr}$ committed[addr][h]. hop 0 total committed matches committed sum.	✓	✓	
CF-16	HL_hop1TotalCommittedMatchesCommittedSum	hop 1 total committed matches committed sum.	✓	✓	
CF-16	HL_hop2TotalCommittedMatchesCommittedSum	hop 2 total committed matches committed sum.	✓	✓	
CF-18	HL_cappedDemandIsHopSumPostFinalize	After finalize, cappedDemand == $\sum_h$ hopStats[h].cappedCommitted	✓	✓	
CF-19	UT_uniqueCommittersOnlyOnFirstCommit	hopStats[h].uniqueCommitters increments exactly once on a participant's first successful commit at hop h, and not on later commits	✓	✓	

Whitelist / Invite Graph (20 properties)

Specification: [specs/Crowdfund/CFInviteGraph.spec](https://specs.crowdfund.com/CFInviteGraph.spec)

Rules passed: 20 Passed | Certora run: [CFInviteGraph.conf](https://certora.com/CFInviteGraph.conf)

Mitigation run: [CFInviteGraph.conf](https://certora.com/CFInviteGraph.conf)

ID	Name	Description	Audit	Mitig	Notes
CF-20	VS_hop0CountBounded	hopStats[0].whitelistCount <= MAX_SEEDS	✓	✓	
CF-21	VS_inviteSlotNonNegative	A whitelisted node's invitesSent never exceeds its slot-derived invite budget	✓	✓	
CF-22	VS_notWhitelistedCantSendInvites	Non-whitelisted nodes have invitesSent == 0	✓	✓	
CF-23	VT_invitesSentNeverDecreases	invitesSent[addr][h] is monotonic non-decreasing	✓	✓	
CF-24	VS_invitesReceivedStackingCapHop0	invitesReceived[addr][h] <= hopConfigs[h].maxInvitesReceived for each hop. Specific focus: invites received stacking cap hop 0.	✓	✓	
CF-24	VS_invitesReceivedStackingCapHop1	invites received stacking cap hop 1.	✓	✓	
CF-24	VS_invitesReceivedStackingCapHop2	invites received stacking cap hop 2.	✓	✓	
CF-25	VS_launchTeamHop1BudgetBounded	launchTeamHop1Used <= LAUNCH_TEAM_HOP1_BUDGET and launchTeamHop2Used <= LAUNCH_TEAM_HOP2_BUDGET. launch team hop 1 budget bounded.	✓	✓	
CF-25	VS_launchTeamHop2BudgetBounded	launch team hop 2 budget bounded.	✓	✓	
CF-26	VS_inviteeHopIsInviterHopPlusOne	Non-launch-team invite edges only create hop h from an inviter whitelisted at hop h - 1	✓	✓	
CF-27	VS_hop0SlotsFromSeedOnly	Hop-0 slots come only from seed insertion and have no invite edge	✓	✓	
CF-28	ST_inviteCreatesParticipationSlot	A successful invite creates a fresh participant slot or increments the existing slot's invitesReceived by exactly 1	✓	✓	
CF-29	VT_participantNodesAppendOnly	participantNodes.length is append-only	✓	✓	
CF-30	ST_addSeedPushesParticipantNodeOnNewSeed	addSeed appends exactly one fresh hop-0 node on success and appends nothing on revert	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
CF-31	ST_invitePushesNodeOnlyOnFirstSlot	invite, launchTeamInvite, and commitWithInvite append a node only when the invitee slot is first created, and reverted invite-like calls never append to participantNodes. Specific focus: invite pushes node only on first slot.	✓	✓	
CF-31	ST_launchTeamInvitePushesNodeOnlyOnFirstSlot	launchTeamInvite pushes node only on first slot.	✓	✓	
CF-31	ST_commitWithInvitePushesNodeOnlyOnFirstSlot	commitWithInvite pushes node only on first slot.	✓	✓	
CF-33	VS_invitesReceivedNonNegative	invitesReceived[addr][h] is always non-negative	✓	✓	
CF-34	ST_addSeedRevertsIfNotActive	addSeed reverts when the sale is not Active	✓	✓	
CF-35	ST_launchTeamInviteRevertsIfNotActive	launchTeamInvite reverts when the sale is not Active	✓	✓	

EIP-712 Invite Nonces (14 properties)

Specification: [specs/Crowdfund/CFSignatureNonce.spec](#)

Rules passed: 14 Passed | **Certora run:** [CFSignatureNonce.conf](#)

Mitigation run: [CFSignatureNonce.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-36	VT_usedNoncesMonotonic	usedNonces[inviter][nonce] is one-way: once true, it never resets	✓	✓	
CF-37	VT_commitWithInviteConsumesNonceOnSuccess	A successful commitWithInvite consumes the specified inviter nonce	✓	✓	
CF-38	VS_nonceUniquenessPerInviter	Used and revoked inviter nonces are permanent and cannot be redeemed again. Specific focus: nonce uniqueness per inviter.	✓	✓	
CF-38	VS_nonceRevocationIsPermanent	nonce revocation is permanent.	✓	✓	
CF-38	ST_revokedNonceBlocksCommitWithInvite	revoked nonce blocks commitWithInvite.	✓	✓	
CF-38	ST_consumedNonceRevertsOnReuse	consumed nonce reverts on reuse.	✓	✓	
CF-39	VS_usedNoncesWritesGated	usedNonces bits flip only through successful commitWithInvite or revokeInviteNonce by the same inviter	✓	✓	
CF-40	VS_commitWithInviteRejectsZeroNonce	commitWithInvite rejects nonce == 0	✓	✓	
CF-41	ST_revokeInviteNonceRejectsZero	revokeInviteNonce(0) reverts	✓	✓	
CF-42	ST_expiredSignaturesRevert	Expired signed invites revert at redemption time	✓	✓	
CF-44	ST_inviteSlotCheckAtRedemptionTime	Invite slot availability is checked at redemption time; exhausted invite budget blocks commitWithInvite	✓	✓	
CF-45	UT_revokeOnlyConsumesSelfNonce	revokeInviteNonce only consumes the caller's own nonce and does not affect other inviter/nonce pairs	✓	✓	
CF-46	UT_commitWithInviteRequiresInviterWhitelisted	commitWithInvite requires the inviter to be whitelisted at fromHop	✓	✓	
CF-47	UT_nonceConsumptionIsolatedToInviter	Consuming a nonce for one inviter does not change the same nonce for another inviter	✓	✓	

Timing / Phase (31 properties)

Specification: [specs/Crowdfund/CFTimingInvariants.spec](#), [specs/Crowdfund/CFFinalization.spec](#), [specs/Crowdfund/CFStateMachine.spec](#)

Rules passed: 31 Passed | **Certora run:** [CFTimingInvariants.conf](#), [CFFinalization.conf](#), [CFStateMachine.conf](#)

Mitigation run: [CFTimingInvariants.conf](#), [CFFinalization.conf](#), [CFStateMachine.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-48	ST_commitRevertsAfterDeadline	commit and commitWithInvite revert before windowStart and after windowEnd. commit reverts after deadline.	✓	✓	
CF-48	ST_commitWithInviteRevertsAfterDeadline	commitWithInvite reverts after deadline.	✓	✓	
CF-48	ST_commitRevertsBeforeOpen	commit reverts before open.	✓	✓	
CF-48	ST_commitWithInviteRevertsBeforeOpen	commitWithInvite reverts before open.	✓	✓	
CF-49	ST_addSeedRevertsAfterWeek1	addSeed and launchTeamInvite revert outside the launch-team week-1 window. addSeed reverts after week 1.	✓	✓	
CF-49	ST_launchTeamInviteRevertsAfterWeek1	launchTeamInvite reverts after week 1.	✓	✓	
CF-49	UT_addSeedSucceedsDuringWeek1WhenValid	addSeed succeeds during week 1 when valid.	✓	✓	
CF-49	UT_launchTeamInviteWeek1BoundaryWhenValid	launchTeamInvite week 1 boundary when valid.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
	d				
CF-50	ST_inviteRevertsAfterDeadline	invite reverts after the commitment deadline	✓	✓	
CF-51	VS_claimDeadlineMatchesFinalizedAtOnSuccessPath	On success-path finalization, claimDeadline == finalizedAt + CLAIM_DEADLINE_DURATION.claim deadline matches finalized at on success path.	✓	✓	
CF-51	VT_finalizeSuccessSetsClaimDeadline	finalize success sets claim deadline.	✓	✓	
CF-52	VT_finalizeRefundModeLeavesClaimDeadlineUnset	Refund-mode finalization leaves claimDeadline unset	✓	✓	
CF-53	VT_finalizeSetsTimestamp	Success-path finalization sets finalizedAt == block.timestamp	✓	✓	
CF-54	ST_finalizeIsPhaseTransition	finalize is one-shot and moves the sale out of Active when it succeeds. finalize is phase transition.	✓	✓	
CF-54	ST_finalizationsIsOneShot	finalization is one shot.	✓	✓	
CF-55	ST_finalizationsIsOneShotAfterCancel	Finalized and Canceled are mutually exclusive terminal phases: finalize reverts after cancellation, and cancel reverts after finalization.finalization is one shot after cancel.	✓	✓	
CF-55	ST_finalizedAndCanceledMutuallyExclusive_fromFinalized	finalized and canceled mutually exclusive from finalized.	✓	✓	
CF-55	ST_finalizedAndCanceledMutuallyExclusive_fromCanceled	finalized and canceled mutually exclusive from canceled.	✓	✓	
CF-56	ST_cancellationsIsOneShot	cancel is one-shot	✓	✓	
CF-58	ST_commitRevertsAfterFinalizationOrCancellation	Active-only entry points remain disabled after finalization or cancellation. commit reverts after finalization or cancellation.	✓	✓	
CF-58	ST_mustRevertAfterPhaseChange_fromFinalized	must revert after phase change from finalized.	✓	✓	
CF-58	ST_mustRevertAfterPhaseChange_fromCanceled	must revert after phase change from canceled.	✓	✓	
CF-59	ST_claimRevertsIfNotFinalized	claim reverts before finalization, in refund mode, or after the caller has already claimed. claim reverts if not finalized.	✓	✓	
CF-59	ST_claimRevertsInRefundMode	claim reverts in refund mode.	✓	✓	
CF-59	ST_claimRevertsIfAlreadyClaimed	claim reverts if already claimed.	✓	✓	
CF-60	ST_claimRefundRevertsIfNotEligible	claimRefund reverts when refunds are unavailable or the caller has already claimed.claimRefund reverts if not eligible.	✓	✓	
CF-60	ST_claimRefundRevertsIfAlreadyClaimed	claimRefund reverts if already claimed.	✓	✓	
CF-61	VT_armLoadedIsPermanent	armLoaded is permanent once set	✓	✓	
CF-62	VT_claimedFlagsIsPermanent	claimed[user] is permanent once set	✓	✓	
CF-63	VT_phaseOnlyProgressesForward	Phase transitions follow Active → {Active, Finalized, Canceled}, Finalized → Finalized, and Canceled → Canceled	✓	✓	
CF-64	VT_refundModeIsIrreversible	refundMode is irreversible once set	✓	✓	

Finalization / Settlement Accounting (14 properties)

Specification: [specs/Crowdfund/CFFinalization.spec](#), [specs/Crowdfund/CFAccounting.spec](#), [specs/Crowdfund/CFAllocation.spec](#), [specs/Crowdfund/CFElasticWaterfall.spec](#)

Rules passed: 14 Passed | **Certora run:** [CFFinalization.conf](#), [CFAccounting.conf](#), [CFAllocation.conf](#), [CFElasticWaterfall.conf](#)

Mitigation run: [CFFinalization.conf](#), [CFAccounting.conf](#), [CFAllocation.conf](#), [CFElasticWaterfall.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-65	VS_refundModeIsFinalizedState	refundMode ⇒ phase == Finalized	✓	✓	
CF-66	VS_successPathHasNonZeroAllocation	Success-path finalized states have totalAllocatedUsdc > 0	✓	✓	
CF-67	ST_saleSizeIsBinary	Success-path saleSize is BASE_SALE or MAX_SALE only	✓	✓	
CF-68	VS_elasticExpansionMatchesSpecTrigger	cappedDemand >= ELASTIC_TRIGGER ⇔ saleSize == MAX_SALE; otherwise success path uses BASE_SALE	✓	✓	
CF-69	ST_elasticNoRefundMode	Elastic expansion cannot end in refund mode. Specific focus: elastic no refund mode.	✓	✓	
CF-69	VS_expansionGuaranteesSuccess	expansion guarantees success.	✓	✓	
CF-70	ST_refundModeAllUsdcAvailableForRefunds	Refund-mode finalization keeps all escrowed USDC in the contract and transfers no USDC to treasury. Specific focus: refund mode all USDC available for refunds.	✓	✓	
CF-70	VT_finalizeRefundPathTransfersNoUsdcToTreasury	finalize refund path transfers no USDC to treasury.	✓	✓	
CF-70	HL_finalizeRefundPathKeepsAllEscrowedUsdc	finalize refund path keeps all escrowed USDC.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
CF-71	HL_finalizeSuccessLeavesRefundPool	Success-path finalization transfers no more than totalAllocatedUsdc to treasury and retains the refund pool plus rounding dust. Specific focus: finalize success leaves refund pool.	✓	✓	
CF-71	VT_finalizeSuccessTransfersExactTreasuryPush	finalize success transfers exact treasury push.	✓	✓	
CF-71	HL_finalizeSuccessLeavesExactRefundPool	finalize success leaves exact refund pool.	✓	✓	
CF-72	VS_refundPathsZeroOutAllocations	refundMode finalization zeroes totalAllocatedUsdc, totalAllocatedArm, and claimDeadline	✓	✓	
CF-73	VS_canceledPathHasNoAllocationsOrArmTransfers	Canceled states have no allocations, no claim deadline, and no ARM transfers	✓	✓	

Allocation / Waterfall / Rounding (47 properties)

Specification: [specs/Crowdfund/CFAllocation.spec](#), [specs/Crowdfund/CFElasticWaterfall.spec](#), [specs/Crowdfund/CFRoundingPrecision.spec](#), [specs/Crowdfund/CFAccounting.spec](#), [specs/Crowdfund/CFClaimSettlement.spec](#)

Rules passed: 47 Passed | **Certora run:** [CFAllocation.conf](#), [CFElasticWaterfall.conf](#), [CFRoundingPrecision.conf](#), [CFAccounting.conf](#), [CFClaimSettlement.conf](#)

Mitigation run: [CFAllocation.conf](#), [CFElasticWaterfall.conf](#), [CFRoundingPrecision.conf](#), [CFAccounting.conf](#), [CFClaimSettlement.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-74	VS_acceptedUsdcNeverExceedsEffective	Accepted USDC per hop never exceeds the slot's effectiveCap or cappedCommitted. Specific focus: accepted USDC never exceeds effective.	✓	✓	
CF-74	VS_acceptedNeverExceedsCappedCommitted	accepted never exceeds capped committed.	✓	✓	
CF-75	VS_overCapExcessIsRefund	Over-cap commitment excess is included in the refund	✓	✓	
CF-76	VS_proRataFloorPerParticipantPerHop	Oversubscribed pro-rata allocation uses floor(cappedCommitted × ceiling / demand). Specific focus: pro rata floor per participant per hop.	✓	✓	
CF-76	VS_ceilingBoundsProRata	ceiling bounds pro rata.	✓	✓	
CF-77	HL_undersubscribedHopFullAllocation	Undersubscribed hops allocate exactly cappedCommitted with no pro-rata rounding	✓	✓	
CF-78	HL_allocPlusRefundEqualsCommittedPerHop	Per-hop and aggregate refund accounting is well-formed: allocUsdc + refundUsdc == committed, refunds are non-negative, and refunds never exceed committed USDC. Specific focus: alloc plus refund equals committed per hop.	✓	✓	
CF-78	HL_aggregateAllocPlusRefundEqualsTotalCommitted	aggregate alloc plus refund equals total committed.	✓	✓	
CF-78	UT_allocPlusRefundEqualsCommitted	alloc plus refund equals committed.	✓	✓	
CF-78	HL_theoreticalSettlementIdentityPerAddress	theoretical settlement identity per address.	✓	✓	
CF-78	VS_refundNonNegative	refund non negative.	✓	✓	
CF-79	HL_computeAllocationIsDeterministic	computeAllocation is deterministic in a fixed finalized state	✓	✓	
CF-80	HL_computeAllocationMatchesPerHopSum	computeAllocation(addr) equals the sum of computeAllocationAtHop(addr, h) across the three hops. Specific focus: compute allocation matches per hop sum.	✓	✓	
CF-80	HL_perHopAllocationSumsToTotal	per hop allocation sums to total.	✓	✓	
CF-81	HL_cvlModelMatchesContractPerHop	The CVL per-hop allocation model matches the contract preview for ARM and refund. Specific focus: cvl model matches contract per hop.	✓	✓	
CF-81	HL_perHopModelMatchesContract	per hop model matches contract.	✓	✓	
CF-82	ST_totalAllocatedBoundedBySaleSize	totalAllocatedUsdc ≤ saleSize and totalAllocatedArm ≤ saleSize × 1e12. Specific focus: total allocated bounded by sale size.	✓	✓	
CF-82	VS_allocatedArmBoundedByMaxSalePostFinalize	allocated ARM bounded by max sale post finalize.	✓	✓	
CF-82	ST_allocatedArmDoesNotExceedSaleSize	allocated ARM does not exceed sale size.	✓	✓	
CF-83	HL_armConversionIsExact	Per-participant and aggregate ARM allocation exactly equals accepted USDC scaled by 1e12. ARM conversion is exact.	✓	✓	
CF-83	HL_armUsdcRatioExactPerParticipant	Per-participant and aggregate ARM allocation exactly equals accepted USDC scaled by 1e12. ARM USDC ratio exact per participant.	✓	✓	
CF-83	ST_armUsdcRatioExactAggregate	ARM USDC ratio exact aggregate.	✓	✓	
CF-83	VS_allocatedArmMatchesAllocatedUsdcPostFinalize	allocated ARM matches allocated USDC post finalize.	✓	✓	
CF-84	ST_finalDemandMatchesHopCappedPostFinalizeHop0	Final stored hop demands equal the corresponding hop capped commitments. Specific focus: final demand matches	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
		hop capped post finalize hop 0.			
CF-84	ST_finalDemandMatchesHopCappedPostFinalize_Hop1	final demand matches hop capped post finalize hop 1.	✓	✓	
CF-84	ST_finalDemandMatchesHopCappedPostFinalize_Hop2	final demand matches hop capped post finalize hop 2.	✓	✓	
CF-84	VS_finalDemandMatchesHopCappedDemandPostFinalize_Hop0	final demand matches hop capped demand post finalize hop 0.	✓	✓	
CF-84	VS_finalDemandMatchesHopCappedDemandPostFinalize_Hop1	final demand matches hop capped demand post finalize hop 1.	✓	✓	
CF-84	VS_finalDemandMatchesHopCappedDemandPostFinalize_Hop2	final demand matches hop capped demand post finalize hop 2.	✓	✓	
CF-85	HL_waterfallModelMatchesContract	Stored final ceilings match the waterfall model: hop 0 uses the post-floor pool, hop 0 leftovers roll into hop 1, hop 1 leftovers roll into hop 2. Specific focus: waterfall model matches contract.	✓	✓	
CF-85	HL_hop0LeftoverRollsIntoHop1Ceiling	hop 0 leftover rolls into hop 1 ceiling.	✓	✓	
CF-85	HL_hop1LeftoverRollsIntoHop2Ceiling	hop 1 leftover rolls into hop 2 ceiling.	✓	✓	
CF-85	VS_allocatedUsdcMatchesStoredDemandAndCeilingsPostFinalize	allocated USDC matches stored demand and ceilings post finalize.	✓	✓	
CF-86	HL_rolloverIsForwardOnly	Rollover is forward-only; hop 0 receives no backward rollover and hop 2 always keeps at least its reserved floor. Specific focus: rollover is forward only.	✓	✓	
CF-86	HL_waterfallOrderIsFixed	waterfall order is fixed.	✓	✓	
CF-86	VS_hop2FloorIsAlwaysReserved	hop 2 floor is always reserved.	✓	✓	
CF-87	HL_maxSaleHop0CeilingExceedsMinSale	At MAX_SALE, the hop-0 ceiling exceeds MIN_SALE; at BASE_SALE, it stays below MIN_SALE. Specific focus: max sale hop 0 ceiling exceeds min sale.	✓	✓	
CF-87	UT_baseSaleHop0CeilingBelowMinSale	base sale hop 0 ceiling below min sale.	✓	✓	
CF-88	UT_hop2AbsorbsFullSaleWhenEarlierHopsAreEmpty	If hop 0 and hop 1 demand are zero, hop 2 absorbs the full sale	✓	✓	
CF-89	UT_overlappingBaseCeilingsSumTo115PctOfAvailable	Hop-0 and hop-1 base ceilings sum to 115% of the available post-floor pool. Specific focus: overlapping base ceilings sum to 115 pct of available.	✓	✓	
CF-89	HL_baseCeilingsUseAvailablePool	base ceilings use available pool.	✓	✓	
CF-90	VS_dustAccumulatesAsUnsoldArm	Rounding dust is non-negative, unsold ARM remains sweepable, and no participant is over-allocated or under-refunded by floor/rounding effects. Specific focus: dust accumulates as unsold ARM.	✓	✓	
CF-90	VS_dustNonNegativeAndBounded	dust non negative and bounded.	✓	✓	
CF-90	VS_armAmountFloorsNeverOverReceives	ARM amount floors never over receives.	✓	✓	
CF-90	VS_hopCeilingNotFullyAllocatedWithRounding	hop ceiling not fully allocated with rounding.	✓	✓	
CF-90	VS_roundingDustBoundedPerParticipant	rounding dust bounded per participant.	✓	✓	
CF-90	VS_refundRoundsUpNeverUnderRefunded	refund rounds up never under refunded.	✓	✓	

Claims / Refunds / Sweeps (19 properties)

Specification: [specs/Crowdfund/CFClaimSettlement.spec](#), [specs/Crowdfund/CFAccounting.spec](#), [specs/Crowdfund/CFCancelInvariants.spec](#), [specs/Crowdfund/CFTimingInvariants.spec](#), [specs/Crowdfund/CFAccountingSharedInvariants.spec](#)

Rules passed: 19 Passed | **Certora run:** [CFClaimSettlement.conf](#), [CFAccounting.conf](#), [CFCancelInvariants.conf](#), [CFTimingInvariants.conf](#)

Mitigation run: [CFClaimSettlement.conf](#), [CFAccounting.conf](#), [CFCancelInvariants.conf](#), [CFTimingInvariants.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-91	VT_totalArmTransferredIsMonotonic	totalArmTransferred is monotonic and, after success-path finalization, bounded by totalAllocatedArm. Specific focus: total ARM transferred is monotonic.	✓	✓	
CF-91	VS_armTransferredBoundedByAllocatedPostFinalize	ARM transferred bounded by allocated post finalize.	✓	✓	
CF-92	VT_claimTransfersExactEntitlementsWithinDeadline	Before claimDeadline, claim(delegate) transfers exactly the computed ARM and refund amounts and increments totalArmTransferred by the ARM delta	✓	✓	
CF-93	VT_claimPostDeadlinePaysRefundOnly	After claimDeadline, claim transfers refund only and leaves ARM balances and totalArmTransferred unchanged. Specific focus: claim post deadline pays refund only.	✓	✓	
CF-93	VT_claimAfterDeadlineTransfersRefundOnly	claim after deadline transfers refund only.	✓	✓	
CF-93	VT_claimAfter3YearDeadlineTransfersNoArm	claim after the 3-year deadline transfers no ARM.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
CF-94	VT_claimRefundPaysExactCommittedWhenSolvent	claimRefund in refund mode or canceled state pays exactly the caller's full committed USDC when solvent. Specific focus: claimRefund pays exact committed when solvent.	✓	✓	
CF-94	VT_refundModeClaimRefundPaysExactCommitted	refund mode claimRefund pays exact committed.	✓	✓	
CF-94	VT_canceledClaimRefundPaysExactCommitted	canceled claimRefund pays exact committed.	✓	✓	
CF-95	VT_cancelRefundPaysExactCommittedWhenSolvent	Canceled-state claimRefund pays exactly the caller's full committed USDC when solvent and sets claimed[msg.sender]	✓	✓	
CF-96	ST_claimPreDeadlineWithArmDueNeedsDelegate	Pre-deadline claim with positive ARM due requires delegate != address(0)	✓	✓	
CF-97	ST_claimPreDeadlineRevertsWhenInsolvent	Pre-deadline insolvent claims revert	✓	✓	
CF-98	ST_cancelOnlySecurityCouncil	cancel succeeds only for securityCouncil while Active and immediately moves the sale to Canceled. Specific focus: cancel only Security Council.	✓	✓	
CF-98	ST_cancelTakesEffectImmediately	cancel takes effect immediately.	✓	✓	
CF-99	VT_canceledPhasesIsSticky	Once canceled, later calls cannot escape the Canceled phase	✓	✓	
CF-100	ST_cancelRefundCannotBeClaimedTwice	Cancel refunds are one-shot	✓	✓	
CF-101	VT_withdrawUnallocatedArmBeforeClaimDeadlineLeavesOwedArm	withdrawUnallocatedArm before claimDeadline leaves exactly totalAllocatedArm - totalArmTransferred ARM owed in the contract and sends only the excess to treasury	✓	✓	
CF-102	VT_withdrawUnallocatedArmFullySweepsWhenNoArmsOwed	withdrawUnallocatedArm after deadline, cancellation, or refund mode sweeps all remaining ARM to treasury	✓	✓	
CF-103	VT_withdrawUnallocatedArmTransfersExactSweep	withdrawUnallocatedArm transfers exactly the sweepable ARM and does not mutate totalArmTransferred	✓	✓	

Settlement State / Shared Accounting Invariants (5 properties)

Specification: [specs/Crowdfund/CFAccounting.spec](#), [specs/Crowdfund/CFAccountingSharedInvariants.spec](#)

Rules passed: 5 Passed | **Certora run:** [CFAccounting.conf](#)

Mitigation run: [CFAccounting.conf](#)

ID	Name	Description	Audit	Mitig	Notes
CF-104	VS_activePhaseHasNoSettlementState	In Active phase, settlement fields remain empty: no sale size, allocations, claim deadline, finalized timestamp, transferred ARM, or refund mode	✓	✓	
CF-105	VS_activeLiveCappedDemandBoundedByCommitted	While Active, live capped demand is bounded by committed demand	✓	✓	
CF-106	VS_boundedWhitelistedSlotsAppearInNodeSet	Bounded participant-node views include all bounded whitelisted slots	✓	✓	
CF-107	VS_boundedPreviewArmSumWithinAggregatePostFinalize	The bounded sum of preview ARM entitlements is at most totalAllocatedArm, and claimed previews cover totalArmTransferred. Specific focus: bounded preview ARM sum within aggregate post finalize.	✓	✓	
CF-107	VS_boundedClaimedPreviewArmCoversTransferredPostFinalize	bounded claimed preview ARM covers transferred post finalize.	✓	✓	

ArmadaGovernor.sol

Initialisation / Admin Latches (32 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_admin.spec](#).

Rules passed: 32 Passed | **Certora run:** [ArmadaGovernorAdmin.conf](#)

Mitigation run: [ArmadaGovernorAdmin.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AG-1	ST_adminInitializeNotReplayable	initialize is callable at most once	✓	✓	
AG-2	VS_adminArmTokenImmutable	armToken, timelock, and treasuryAddress are immutable after initialization. Specific focus: admin ARM token immutable.	✓	✓	
AG-2	VS_adminTimelockImmutable	admin timelock immutable.	✓	✓	
AG-2	VS_adminTreasuryImmutable	admin treasury immutable.	✓	✓	
AG-3	VT_adminExcludedAddressesLockedOneWay	excludedAddressesLocked is one-way; a second setExcludedAddresses call reverts. Specific focus: admin excluded addresses locked one way.	✓	✓	
AG-3	ST_adminExcludedAddressesLockedSecondCallReverts	admin excluded addresses locked second call reverts.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AG-3	VS_stateExcludedAddressesLockPermanent	state excluded addresses lock permanent.	✓	✓	
AG-4	VS_adminExcludedListImmutableAfterLock	_excludedFromQuorum length is immutable after the exclusion list is locked	✓	✓	
AG-5	VT_adminCrowdfundAddressLockedOneWay	crowdfundAddressLocked, stewardContractLocked, and windDownContractSet are one-way latches. Specific focus: admin crowdfund address locked one way.	✓	✓	
AG-5	VT_adminStewardContractLockedOneWay	admin steward contract locked one way.	✓	✓	
AG-5	VT_adminWindDownContractSetOneWay	admin wind-down contract set one way.	✓	✓	
AG-6	VS_adminCrowdfundAddressImmutableAfterLock	crowdfundAddress, stewardContract, and windDownContract are immutable after their latches are set. Specific focus: admin crowdfund address immutable after lock.	✓	✓	
AG-6	VS_adminStewardContractImmutableAfterLock	admin steward contract immutable after lock.	✓	✓	
AG-6	VS_adminWindDownContractImmutableAfterSet	admin wind-down contract immutable after set.	✓	✓	
AG-7	VT_adminDeployerMonotonicCleared	deployer == address(0) is terminal	✓	✓	
AG-8	VT_adminExcludedAddressesLockedRequiresDeployer	Deployer-only latch flips occur only through the corresponding deployer-only setters. Specific focus: admin excluded addresses locked requires deployer.	✓	✓	
AG-8	VT_adminCrowdfundAddressRequiresDeployer	admin crowdfund address requires deployer.	✓	✓	
AG-8	VT_adminStewardContractRequiresDeployer	admin steward contract requires deployer.	✓	✓	
AG-9	VT_adminClearDeployerRequiresDeployer	clearDeployer can only be called by the current deployer	✓	✓	
AG-10	VT_adminSecurityCouncilRequiresTimelock	securityCouncil changes only through timelock governance, active deployer bootstrap, or veto-ratification resolution	✓	✓	
AG-11	VT_adminWindDownContractRequiresTimelock	windDownContractSet can flip only through setWindDownContract by the timelock	✓	✓	
AG-12	VT_adminExtendedSelectorRequiresTimelock	extendedSelectors and standardSelectors mutate only through their add/remove functions by the timelock. Specific focus: admin extended selector requires timelock.	✓	✓	
AG-12	VT_adminStandardSelectorRequiresTimelock	admin standard selector requires timelock.	✓	✓	
AG-13	ST_adminAfterClearDeployerAllDeployerSettersRevert	After clearDeployer, all deployer-only setters and clearDeployer itself revert. Specific focus: admin after clear deployer all deployer setters revert.	✓	✓	
AG-13	ST_adminSetCrowdfundAddressAfterClearReverts	admin set crowdfund address after clear reverts.	✓	✓	
AG-13	ST_adminSetStewardContractAfterClearReverts	admin set steward contract after clear reverts.	✓	✓	
AG-13	ST_adminClearDeployerAfterClearReverts	admin clear deployer after clear reverts.	✓	✓	
AG-14	VS_adminConstantsImmutable	PROPOSAL_THRESHOLD, QUEUE_GRACE_PERIOD, QUORUM_FLOOR, and QUIET_PERIOD_DURATION are immutable constants	✓	✓	
AG-15	ST_classificationSetExcludedRejectsZero	Excluded addresses reject address(0) and treasuryAddress. Specific focus: classification set excluded rejects zero.	✓	✓	
AG-15	ST_classificationSetExcludedRejectsTreasury	classification set excluded rejects treasury.	✓	✓	
AG-16	VS_classificationExcludedListCleanliness	Every stored excluded address is non-zero and not the treasury	✓	✓	
AG-17	VT_lifecycleClearDeployerDoesNotDisturbLatches	clearDeployer can remove bootstrap power without changing excluded-address, crowdfund, steward, or wind-down latches or stored addresses	✓	✓	

Proposal Bookkeeping (13 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_bookkeeping.spec](#),
[specs/Governance/ArmadaGovernor/ArmadaGovernor_proposals.spec](#).

Rules passed: 13 Passed | **Certora run:** [ArmadaGovernorBookkeeping.conf](#), [ArmadaGovernorProposals.conf](#)

Mitigation run: [ArmadaGovernorBookkeeping.conf](#), [ArmadaGovernorProposals.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AG-18	VT_bookkeepingExistingProposalMetadataFrozen	Existing proposal metadata is frozen after creation: proposer, type, vote window, execution delay, snapshot block, eligible supply, and quorum bps	✓	✓	
AG-19	VT_bookkeepingProposalCountChangesOnlyOnSuccessfulCreators	proposalCount is monotonic, increments by at most one per call, and increments only on successful proposal creation. Specific focus: bookkeeping proposal count changes only on successful creators.	✓	✓	
AG-19	VS_stateProposalCountMonotonic	state proposal count monotonic.	✓	✓	
AG-20	VT_lifecycleFailedProposalCreationDoesNotConsumeIDs	Reverted calls do not consume proposal IDs	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AG-21	VS_bookkeepingVoteEndGeVoteStart	voteEnd >= voteStart for every existing proposal	✓	✓	
AG-22	VT_proposalsSnapshotPreviousBlockOnCreation	Proposal creation snapshots voting power at block.number - 1	✓	✓	
AG-23	ST_votesProposeRequiresThreshold	Proposal threshold checks use the previous-block snapshot. Specific focus: votes propose requires threshold.	✓	✓	
AG-24	VT_proposalsExistingProposalTypesSnapshotStable	Stored proposal type and timing snapshots remain committed after selector, SC, or proposal-param admin edits. Specific focus: proposals existing proposal type is snapshot stable.	✓	✓	
AG-24	VT_proposalsExistingProposalTypeAndTimingStayCommitted	proposals existing proposal type and timing stay committed.	✓	✓	
AG-25	UT_classificationEmptyExecutableProposalRevertsWithoutId	Failed executable proposal creation and governance array mismatches do not consume IDs. Specific focus: classification empty executable proposal reverts without ID.	✓	✓	
AG-25	UT_classificationGovernanceArrayMismatchRevertsWithoutId	classification governance array mismatch reverts without ID.	✓	✓	
AG-26	VT_lifecycleTimelockAdminSettersDoNotAffectTallies	Timelock admin setter calls do not mutate proposal vote tallies	✓	✓	
AG-27	ST_proposalsSignalingPayloadRejected	Signaling proposals are text-only and cannot carry execution payloads	✓	✓	

Lifecycle / State Machine (41 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_lifecycle.spec](#),
[specs/Governance/ArmadaGovernor/ArmadaGovernor_part2_lifecycle.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_bookkeeping.spec](#),
[specs/Governance/ArmadaGovernor/ArmadaGovernor_state.spec](#)

Rules passed: 40 Passed, 1 Timeout | Certora run: [ArmadaGovernorLifecycle.conf](#), [ArmadaGovernorLifecyclePart2.conf](#), [ArmadaGovernorState.conf](#)

Mitigation run: [ArmadaGovernorLifecycle.conf](#), [ArmadaGovernorLifecyclePart2.conf](#), [ArmadaGovernorState.conf](#),
[VS_lifecycleCanceledDominatesExecuted](#)

ID	Name	Description	Audit	Mitig	Notes
AG-29	VT_bookkeepingExecutedOnlyViaExecutionPaths	executed is terminal and flips only through execute or resolveRatification on non-reverted calls. Specific focus: bookkeeping executed only via execution paths.	✓	✓	
AG-29	VS_stateExecutedFlagPermanent	state executed flag permanent.	✓	✓	
AG-30	VT_bookkeepingCanceledTransitionsAreGated	canceled flips only through proposer cancel or SC veto, and clears only through veto-denial restoration. Specific focus: bookkeeping canceled transitions are gated.	✓	✓	
AG-30	VS_stateCanceledFlagPermanentExceptRatificationRestore	state canceled flag permanent except ratification restore.	✓	✓	
AG-31	VT_bookkeepingQueuedOnlyViaQueue	queued flips only through queue and never resets	✓	✓	
AG-32	VT_bookkeepingVetoRatificationDeniedOnlyViaDeniedVetoRestore	vetoRatificationDenied is one-way and set only by resolveRatification while restoring the original proposal	✓	✓	
AG-33	ST_lifecycleQueueRequiresSucceeded	queue succeeds only from Succeeded and rejects Signaling and VetoRatification proposals. Specific focus: lifecycle queue requires succeeded.	✓	✓	
AG-33	ST_lifecycleQueueRejectsSignalingAndRatification	lifecycle queue rejects signaling and ratification.	✓	✓	
AG-34	ST_lifecycleExecuteRequiresQueued	execute succeeds only from Queued and rejects Signaling and VetoRatification proposals. Specific focus: lifecycle execute requires queued.	✓	✓	
AG-34	ST_lifecycleExecuteRejectsSignalingAndRatification	lifecycle execute rejects signaling and ratification.	✓	✓	
AG-35	ST_lifecycleCancelRequiresProposer	cancel succeeds only for the proposer, in Pending for normal proposals and Pending or Active for Steward proposals. Specific focus: lifecycle cancel requires proposer.	✓	✓	
AG-35	ST_lifecycleCancelRequiresPendingOrActive	lifecycle cancel requires pending or active.	✓	✓	
AG-36	ST_lifecycleVetoPreconditions	veto requires the current non-zero SC, Queued state, and no prior denied-veto restoration	✓	✓	
AG-37	ST_lifecycleExecuteRejectsAlreadyExecuted	Already executed proposals cannot be executed again	✓	✓	
AG-38	VT_lifecycleWindDownActiveOneWay	windDownActive is a one-way latch and can be set only by the registered wind-down contract. Specific focus: lifecycle wind-down active one way.	✓	✓	
AG-38	ST_lifecycleSetWindDownActiveRequiresWindDownContract	lifecycle set wind-down active requires wind-down contract.	✓	✓	
AG-38	ST_lifecycleWindDownOnlyThroughRegisteredContract	lifecycle wind-down only through registered contract.	✓	✓	
AG-38	VS_stateWindDownActivePermanent	state wind-down active permanent.	✓	✓	
AG-39	ST_lifecycleProposeRejectedWhenWindDown	When windDownActive, propose and proposeStewardSpend always revert. Specific focus: lifecycle propose rejected when wind-down.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AG-39	ST_lifecycleProposeStewardSpendRejectedWhenWindDown	lifecycle propose stewardSpend rejected when wind-down.	✓	✓	
AG-39	ST_stateWindDownBlocksGovernanceProposals	state wind-down blocks governance proposals.	✓	✓	
AG-39	ST_stateWindDownBlocksStewardProposals	state wind-down blocks steward proposals.	✓	✓	
AG-40	ST_lifecycleWindDownDoesNotCancelInFlight	Activating wind-down does not retroactively cancel in-flight proposals	✓	✓	
AG-41	ST_lifecycleTimelockCannotDirectlyActivateWindDown	Timelock cannot directly activate wind-down unless it is the registered wind-down contract	✓	✓	
AG-44	ST_lifecycleProposeRejectedInQuietPeriod	During the post-crowdfund quiet period, both ordinary proposals and steward proposals revert. Specific focus: lifecycle propose rejected in quiet period.	✓	✓	
AG-44	ST_lifecycleProposeStewardSpendRejectedInQuietPeriod	lifecycle propose stewardSpend rejected in quiet period.	✓	✓	
AG-45	UT_lifecycleQuietPeriodConstant	QUIET_PERIOD_DURATION == 7 days	✓	✓	
AG-46	ST_lifecycleGraceExpiryDefeats	Non-signaling, non-steward proposals past voteEnd + QUEUE_GRACE_PERIOD without queue/cancel/execute are Defeated	✓	✓	
AG-47	UT_lifecycleQueueGracePeriodConstant	QUEUE_GRACE_PERIOD == 14 days	✓	✓	
AG-48	VT_lifecycleFailedQueueAtomic	Failed queue and failed execute preserve queued/executed/canceled flags and vote tallies. Specific focus: lifecycle failed queue atomic.	✓	✓	
AG-48	VT_lifecycleFailedExecuteRetryable	lifecycle failed execute retryable.	✓	✓	
AG-49	ST_lifecycleNormalExecutableLifecycleOrdering	Successful execution starts from Queued and marks the proposal executed	✓	✓	
AG-50	VS_lifecycleNonStewardSuccessRequiresQuorumAndForMajority	Normal executable proposals reach Succeeded only with quorum and strict forVotes > againstVotes	✓	✓	
AG-53	VS_lifecycleCanceledDominatesExecuted	Canceled proposals report Canceled and proposer cancellation of a pending proposal succeeds. Specific focus: lifecycle canceled dominates executed.	Timeout	✓	
AG-54	ST_lifecycleSignalingTerminalNonExecutable	Signaling proposals cannot be queued, executed, vetoed, or used for binding protocol side effects. Specific focus: lifecycle signaling terminal non executable.	✓	✓	
AG-54	ST_lifecycleSignalingCannotBeVetoed	lifecycle signaling cannot be vetoed.	✓	✓	
AG-54	HL_lifecycleSignalingNoBindingProtocolEffect	lifecycle signaling no binding protocol effect.	✓	✓	
AG-54	ST_lifecycleSignalingQueueReverts	lifecycle signaling queue reverts.	✓	✓	
AG-54	ST_lifecycleSignalingExecuteReverts	lifecycle signaling execute reverts.	✓	✓	
AG-56	ST_lifecycleRatificationNeverQueued	Veto-ratification proposals can never be queued	✓	✓	
AG-57	ST_lifecycleExecuteNotReentrantBackToBack	Back-to-back execute on an executed proposal is rejected, modeling non-reentrant/retry-safe behavior	✓	✓	

Voting / Quorum / Threshold (17 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_votes.spec](#),

[specs/Governance/ArmadaGovernor/ArmadaGovernor_part2_votes.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_state.spec](#)

Rules passed: 17 Passed | **Certora run:** [ArmadaGovernorVotes.conf](#), [ArmadaGovernorVotesPart2.conf](#)

Mitigation run: [ArmadaGovernorVotes.conf](#), [ArmadaGovernorVotesPart2.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AG-58	VT_votesFirstVoteMarksParticipationAndAddsWeight	First-time castVote adds exactly the voter's snapshot weight to the chosen bucket and marks participation	✓	✓	
AG-59	VT_votesVoteSwitchUpdatesChoiceAtomically	Vote switching is atomic: old bucket loses exactly the snapshot weight, new bucket gains it, and total participation is unchanged. Specific focus: votes vote switch updates choice atomically.	✓	✓	
AG-59	VT_votesVoteTallyDeltaConsistency	votes vote tally delta consistency.	✓	✓	
AG-60	VT_votesSameVoteRevertsWithoutMutation	Repeating the same vote choice reverts without mutating tallies	✓	✓	
AG-61	VT_votesCanceledProposalFreezesVotes	Canceled proposals freeze votes, hasVoted, and voteChoice	✓	✓	
AG-62	VT_votesTotalParticipationMonotonic	Total participation is monotonic per proposal. Specific focus: votes total participation monotonic.	✓	✓	
AG-62	VS_stateTotalParticipationNonDecreasing	state total participation non decreasing.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AG-63	VT_votesTallyFrozenOutsideWindow	Vote tallies are frozen outside [voteStart, voteEnd]	✓	✓	
AG-64	ST_votesCastVotePreconditions	castVote success requires an existing, uncanceled proposal, in-window timestamp, support <= 2, and positive snapshot weight	✓	✓	
AG-65	VS_votesVoteChoiceBounded	voteChoice is always in {0,1,2} for voters and hasVoted is one-way. Specific focus: votes vote choice bounded.	✓	✓	
AG-65	VT_votesHasVotedOneWay	votes has voted one way.	✓	✓	
AG-66	HL_votesVoteWeightFromSnapshot	Vote weight comes from getPastVotes(voter, snapshotBlock), not live voting power	✓	✓	
AG-69	HL_votesQuorumStableAcrossEnvs	quorum(id) is stable across repeated queries and equals $\max(\text{snapshotEligibleSupply} \times \text{snapshotQuorumBps} / 10000, \text{QUORUM_FLOOR})$. Specific focus: votes quorum stable across environments.	✓	✓	
AG-69	HL_votesQuorumFormula	votes quorum formula.	✓	✓	
AG-70	VS_votesSnapshotEligibleSupplyBounded	snapshotEligibleSupply <= getPastTotalSupply(snapshotBlock)	✓	✓	
AG-71	HL_votesQuorumReachedDefinition	_quorumReached is equivalent to forVotes + againstVotes + abstainVotes >= quorum(id)	✓	✓	
AG-73	UT_votesProposalThresholdConstant	PROPOSAL_THRESHOLD == 5_000 × 1e18 and proposalThreshold() returns the same value	✓	✓	

Classification / Timelock Surface / Outflow Feasibility (33 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_classification.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_part2_classification.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_proposals.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_state.spec](#)

Rules passed: 32 Passed, 1 Timeout | **Certora run:** [ArmadaGovernorClassification.conf](#), [ArmadaGovernorClassificationPart2.conf](#), [ArmadaGovernorProposals.conf](#), [ArmadaGovernorState.conf](#)

Mitigation run: [ArmadaGovernorClassification.conf](#), [ArmadaGovernorProposals.conf](#), [ArmadaGovernorState.conf](#)

ArmadaGovernorClassificationPart2: (some rules needs to run independently due to timeouts) :

[1](#),[2](#),[3](#),[4](#),[5](#),[6](#)

ID	Name	Description	Audit	Mitig	Notes
AG-74	VS_classificationExtendedDominatesDualRegistration	Extended selector registration dominates dual registration with Standard	✓	✓	
AG-75	VS_classificationAnyExtendedActionMakesBatchExtended	Any Extended-classified action makes the whole batch Extended	✓	✓	
AG-76	HL_classificationExtendedSelectorForcesExtended	Declared Extended proposals stay Extended; unknown selectors fail closed to Extended. Specific focus: classification extended selector forces extended.	✓	✓	
AG-76	HL_classificationExtendedDeclaredPreserved	classification extended declared preserved.	✓	✓	
AG-76	HL_classificationUnknownSelectorFailsClosed	classification unknown selector fails closed.	✓	✓	
AG-77	UT_classificationAboveFivePercentDistributionBecomesExtended	A distribution strictly above 5% of treasury balance classifies as Extended	✓	✓	
AG-78	UT_classificationNonTimelockUpdateDelayCallDataIgnored	Non-timelock updateDelay-looking calldata is ignored by the timelock-delay cap guard	✓	✓	
AG-79	ST_classificationUpdateDelayExceedingCapReverts	timelock.updateDelay(newDelay) with newDelay > MAX_EXECUTION_DELAY is rejected	✓	✓	
AG-80	HL_classificationSaltEqualsId	proposalSalt(id) == bytes32(id) and salts are injective by proposal id. Specific focus: classification salt equals ID.	✓	✓	
AG-80	HL_classificationSaltInjective	classification salt injective.	✓	✓	
AG-81	ST_classificationProposeStewardSpendRejectsExtendedClassification	proposeStewardSpend rejects calldata that classifies as Extended	✓	✓	
AG-82	HL_classificationStewardProposalStructure	Steward proposals allocate exactly one new ID, are typed Steward, target the treasury, call only stewardSpend, and use zero ETH value	Timeout	Timeout	91% progress
AG-83	ST_classificationProposeStewardSpendRejectsSelfPayment	Steward proposals reject self-payment recipients	✓	✓	
AG-84	ST_classificationStewardProposalCreationIsBudgetAgnosticAndContained	Steward proposal creation does not read treasury budget state	✓	✓	
AG-85	ST_proposalsSameTokenDistributionsAggregateAgainstLimit	Queue-time feasibility aggregates same-token distributions against one effective limit	✓	✓	
AG-86	ST_proposalsDistributionAndStewardSpendShareAggregateLimit	distribute and stewardSpend to the same token share the same aggregate feasibility limit	✓	✓	
AG-87	ST_proposalsQueueFeasibilityUsesEffectiveLimitCeiling	Queue-time feasibility keys off the effective limit ceiling, not temporary available outflow	✓	✓	
AG-88	ST_classificationNonTreasurySpendSelectorsDoNotConsumeTreasuryBudget	Non-treasury spend-like calldata does not consume treasury budget	✓	✓	
AG-89	UT_classificationDifferentTokensUseIndependent	Distinct tokens are checked against independent feasibility	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
	tFeasibilitySums	sums			
AG-90	ST_classificationSingleTokenOutflowAboveEffectiveLimitReverts	A single token outflow, or aggregated same-token outflows, above effective limit reverts. Specific focus: classification single token outflow above effective limit reverts.	✓	✓	
AG-90	ST_classificationAggregatedSameTokenOutflowAboveEffectiveLimitReverts	classification aggregated same token outflow above effective limit reverts.	✓	✓	
AG-91	VS_classificationQueueUsesTimelockHashAndSchedule	queue schedules through the timelock; execute calls timelock.executeBatch and never calls proposal targets directly. Specific focus: classification queue uses timelock hash and schedule.	✓	✓	
AG-91	VS_classificationExecuteUsesTimelockAndAvoidsDirectTargetCalls	classification execute uses timelock and avoids direct target calls.	✓	✓	
AG-92	UT_classificationExecuteForwardsMsgValueThroughTimelock	execute forwards msg.value through timelock execution	✓	✓	
AG-93	VS_classificationVetoUsesTimelockHashAndCancel	veto cancels the queued timelock operation through the timelock	✓	✓	
AG-94	UT_classificationDeniedVetoReadsMinDelayAndReschedules	Restoring a denied veto re-schedules through the timelock	✓	✓	
AG-95	ST_classificationUpgradeToRequiresTimelock	upgradeTo requires timelock authorization	✓	✓	
AG-96	VT_classificationExtendedSelectorMutatorShape	Selector registry mutations have the correct add/remove shape. Specific focus: classification extended selector mutator shape.	✓	✓	
AG-96	VT_classificationStandardSelectorMutatorShape	classification standard selector mutator shape.	✓	✓	
AG-97	ST_classificationSetProposalTypeParamsBound	setProposalTypeParams enforces voting delay, voting period, execution delay, and quorum-bps bounds	✓	✓	
AG-98	VS_classificationExtendedCycleBounded	Extended proposal cycle duration remains strictly below the treasury outflow activation delay	✓	✓	
AG-99	ST_classificationImmutableProposalTypes	VetoRatification, Steward, and Signaling proposal type params are immutable. Specific focus: classification immutable proposal types.	✓	✓	
AG-99	ST_stateImmutableProposalTypeParamsRevert	state immutable proposal type params revert.	✓	✓	

Security Council / Veto Ratification (17 properties)

Specification: [specs/Governance/ArmadaGovernor/ArmadaGovernor_veto.spec](#), [specs/Governance/ArmadaGovernor/ArmadaGovernor_part2_veto.spec](#)

Rules passed: 17 Passed | **Certora run:** [ArmadaGovernorVeto.conf](#), [ArmadaGovernorVetoPart2.conf](#)

Mitigation run: [ArmadaGovernorVeto.conf](#), [ArmadaGovernorVetoPart2.conf](#), [VT_vetoSuccessfulVetoCreatesRatification](#)

ID	Name	Description	Audit	Mitig	Notes
AG-100	ST_vetoOnlySecurityCouncilCanVeto	Only the current non-zero Security Council can successfully veto	✓	✓	
AG-101	ST_vetoEjectedSecurityCouncilCannotVeto	An ejected Security Council cannot veto anything. Specific focus: veto ejected Security Council cannot successfully veto.	✓	✓	
AG-101	ST_vetoRejectsEjectedSC	veto rejects ejected Security Council.	✓	✓	
AG-102	ST_vetoSecurityCouncilHasNoDirectParameterAuthority	Security Council has no direct parameter or governor-admin setter authority outside timelock/deployer authorization. Specific focus: veto Security Council has no direct parameter authority.	✓	✓	
AG-102	ST_vetoSecurityCouncilCannotUseGovernorAdminSetters	veto Security Council cannot use governor admin setters.	✓	✓	
AG-103	VT_vetoSuccessfulVetoCreatesRatification	A successful veto cancels the original proposal, creates exactly one ratification proposal, and populates both ratification mappings. Specific focus: veto successful veto creates ratification.	✓	✓	
AG-104	VT_vetoVetoAtomicity	vetoRatificationId[x] and ratificationOff[y] are bidirectionally consistent	✓	✓	
AG-105	VT_vetoFailedVetoLeavesStateUntouched	Failed veto attempts do not change proposal state, proposal count, or ratification bookkeeping	✓	✓	
AG-106	ST_vetoSingleVetoRule	veto reverts when vetoRatificationDenied is already set for the original proposal	✓	✓	
AG-107	ST_vetoResolveRatificationPreconditions	resolveRatification succeeds only for a linked ratification after vote end and before execution; success marks the ratification executed	✓	✓	
AG-108	VT_vetoFailedResolveLeavesSecurityCouncilUntouched	Resolving a non-ratification proposal reverts without changing the Security Council	✓	✓	
AG-109	ST_vetoDeniedVetoEjectsSCAndRestoresProposal	Majority-against ratification with quorum ejects the vetoing SC if still seated, restores the original proposal, and sets the denied-veto latch. Specific focus: veto denied veto ejects Security Council and restores proposal.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AG-109	ST_vetoMajorityAgainstEjectsSC	veto majority against ejects Security Council.	✓	✓	
AG-110	HL_vetoUpheldVetoKeepsProposalCanceledAndSecurityCouncil	If the community does not deny the veto, the original proposal remains canceled and the SC is preserved	✓	✓	
AG-111	UT_vetoRatificationTieUpholdsVeto	Ratification ties and ratifications without quorum uphold the veto. Specific focus: veto ratification tie upholds veto.	✓	✓	
AG-111	UT_vetoRatificationWithoutQuorumUpholdsVeto	veto ratification without quorum upholds veto.	✓	✓	
AG-112	UT_vetoOnlyTimelockCanZeroSecurityCouncil	Arbitrary callers that are neither timelock nor the active bootstrap deployer cannot zero the Security Council directly	✓	✓	

AdapterRegistry.sol

Adapter Authorization State Machine (15 properties)

Specification: [specs/Governance/AdapterRegistry.spec](#)

Rules passed: 15 Passed | Certora run: [AdapterRegistry.conf](#)

Mitigation run: [AdapterRegistry.conf](#)

ID	Name	Description	Audit	Mitig	Notes
ADR-1	VS_adapterMutualExclusion	authorizedAdapters[a] and withdrawOnlyAdapters[a] are mutually exclusive	✓	✓	
ADR-2	VT_adapterStateChangesRequireTimelock	Any adapter authorization or withdraw-only state change requires msg.sender == timelock	✓	✓	
ADR-3	VS_adapterTimelockImmutable	timelock is immutable	✓	✓	
ADR-4	ST_adapterFromCleanState	A clean adapter can only stay clean or become authorized through authorizeAdapter	✓	✓	
ADR-5	ST_adapterFromAuthorized	An authorized adapter can only stay authorized or become withdraw-only through deauthorizeAdapter	✓	✓	
ADR-6	ST_adapterFromWithdrawOnly	A withdraw-only adapter can only stay withdraw-only, fully deauthorize, or re-authorize through the matching mutator	✓	✓	
ADR-7	VS_adapterValidStates	Every adapter remains in one of the three reachable states: clean, authorized, or withdraw-only	✓	✓	
ADR-8	ST_adapterAuthorizeRevertsOnZeroOrDuplicate	authorizeAdapter reverts for address(0) and already-authorized adapters	✓	✓	
ADR-9	ST_adapterDeauthorizeRevertsWhenNotAuthorized	deauthorizeAdapter reverts unless the adapter is currently authorized	✓	✓	
ADR-10	ST_adapterFullDeauthorizeRevertsWhenNotWithdrawOnly	fullDeauthorizeAdapter reverts unless the adapter is currently withdraw-only	✓	✓	
ADR-11	UT_adapterGettersAreStable	Public adapter mapping getters are stable within a rule context	✓	✓	
ADR-12	VT_adapterAuthorizeDoesNotTouchOtherAddresses	authorizeAdapter(target) does not mutate unrelated adapter slots	✓	✓	
ADR-13	VT_adapterDeauthorizeDoesNotTouchOtherAddresses	deauthorizeAdapter(target) does not mutate unrelated adapter slots	✓	✓	
ADR-14	VT_adapterFullDeauthorizeDoesNotTouchOtherAddresses	fullDeauthorizeAdapter(target) does not mutate unrelated adapter slots	✓	✓	
ADR-15	ST_adapterAuthorizeClearsWithdrawOnly	Successful authorizeAdapter(a) clears withdrawOnlyAdapters[a] and sets authorizedAdapters[a]	✓	✓	

ShieldPauseController.sol

Pause / Wind-Down Controls (18 properties)

Specification: [specs/Governance/ShieldPauseController.spec](#)

Rules passed: 18 Passed | Certora run: [ShieldPauseController.conf](#)

Mitigation run: [ShieldPauseController.conf](#)

ID	Name	Description	Audit	Mitig	Notes
SP-1	VS_shieldWindDownContractSetIfNonZero	windDownContractSet ⇔ windDownContract != address(0)	✓	✓	
SP-2	VS_shieldEmergencyPausedFalsePreWindDown	emergencyPaused() remains false before wind-down activation	✓	✓	
SP-3	HL_shieldEmergencyPausedMatchesComponents	emergencyPaused() ⇔ windDownActive && _paused && block.timestamp < pauseExpiry	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
SP-4	VT_shieldWindDownActiveMonotonic	windDownActive is a one-way latch	✓	✓	
SP-5	VT_shieldWindDownActiveOnlyViaSetter	windDownActive changes only through setWindDownActive	✓	✓	
SP-6	ST_shieldSetWindDownActiveAuth	Successful setWindDownActive requires msg.sender == windDownContract and inactive pre-state	✓	✓	
SP-7	VT_shieldWindDownContractOneShot	Once set, windDownContract remains stable outside setWindDownContract	✓	✓	
SP-8	ST_shieldSetWindDownContractAuth	Successful setWindDownContract(wdc) requires msg.sender == pauseTimelock and wdc != address(0)	✓	✓	
SP-9	ST_shieldSetWindDownContractSecondCallReverts	A second setWindDownContract call reverts	✓	✓	
SP-10	VT_shieldWindDownPauseUsedMonotonic	windDownPauseUsed is a one-way latch	✓	✓	
SP-11	VT_shieldWindDownPauseUsedOnlyViaAuthorizedConsumption	windDownPauseUsed flips only during post-wind-down pauseShields, or during setWindDownActive when an active pre-trigger pause bleeds into wind-down	✓	✓	
SP-12	ST_shieldPauseShieldsAuth	pauseShields requires the current non-zero Security Council and an unpaused state; after wind-down, the one-shot pause must be unused	✓	✓	
SP-13	ST_shieldUnpauseShieldsAuth	unpauseShields requires msg.sender == pauseTimelock and an active pause	✓	✓	
SP-14	ST_shieldPauseShieldsSetsExpiry	Successful pauseShields sets pauseExpiry == block.timestamp + MAX_PAUSE_DURATION and sets _paused	✓	✓	
SP-15	HL_shieldAutoExpiresBeforeWindDown	Before wind-down, shieldsPaused() auto-expires once block.timestamp >= pauseExpiry	✓	✓	
SP-16	ST_shieldUnpauseShieldsAtomicClear	Successful unpauseShields clears _paused and resets pauseExpiry to zero	✓	✓	
SP-17	VS_shieldPostWindDownPauseCountMatchesFlag	The post-wind-down pause-budget consumption counter mirrors windDownPauseUsed	✓	✓	
SP-18	VS_shieldPostWindDownPauseCountBounded	At most one emergency-pause budget consumption can occur	✓	✓	

ArmadaTreasuryGov.sol

Model / Config Foundations (12 properties)

Specification: [specs/Governance/TreasuryGov.spec](#)

Rules passed: 12 Passed | Certora run: [TreasuryGov.conf](#)

Mitigation run: [TreasuryGov.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-1	UT_setupGhostsMatchRawViews	CVL ghost mirrors for outflow config, pending slots, steward budgets, and initialization equal the raw contract view getters	✓	✓	
TG-2	HL_effectiveLimitFormulaMatchesSpec	_effectiveLimit(balance) == max(max(balance × limitBps / 10000, limitAbsolute), floorAbsolute)	✓	✓	
TG-4	VS_activeLimitBpsInRange	Active limitBps is in [1, 10000] for initialized tokens	✓	✓	
TG-5	VS_activeWindowAtLeastOneDay	Active windowDuration >= 1 day for initialized tokens	✓	✓	
TG-6	VS_activeAbsoluteNeverBelowFloor	Active and pending absolute limits are never below floorAbsolute. Specific focus: active absolute never below floor.	✓	✓	
TG-6	VS_pendingAbsoluteNeverBelowFloor	pending absolute never below floor.	✓	✓	
TG-7	VS_authorizedStewardBudgetParamsValid	Authorized steward budgets have limit > 0 and window >= 1 day	✓	✓	
TG-8	VS_pendingLimitBpsInRange	Pending bps and pending window slots satisfy the same range and one-day lower bound when set. Specific focus: pending limit BPS in range.	✓	✓	
TG-8	VS_pendingWindowAtLeastOneDayWhenSet	pending window at least one day when set.	✓	✓	
TG-9	VT_initializedOneWay	initialized[token] is one-way	✓	✓	
TG-10	VT_floorAbsoluteImmutablePostInit	floorAbsolute[token] is immutable after initialization	✓	✓	
TG-11	ST_setAbsoluteRespectsFloor	setOutflowLimitAbsolute rejects values below floorAbsolute	✓	✓	

Outflow History / Rate Limits (16 properties)

Specification: [specs/Governance/TreasuryGov.spec](#)

Rules passed: 16 Passed | Certora run: [TreasuryGov.conf](#)

Mitigation run: [TreasuryGov.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-12	VT_outflowHistoryAppendOnly	Aggregate outflow history length is append-only	✓	✓	
TG-13	VT_outflowRecorderAppendsAtMostOne	Successful aggregate outflow recorders append at most one record per call	✓	✓	
TG-14	VT_stewardHistoryAppendOnly	Steward spend history is append-only, including across removeStewardBudgetToken	✓	✓	
TG-15	VT_outflowRecordFieldsImmutable	Existing outflow record amount and timestamp fields are immutable	✓	✓	
TG-16	VT_newOutflowTimestampMonotonic	Newly appended outflow record timestamps are non-decreasing and equal block.timestamp	✓	✓	
TG-17	HL_recentOutflowWithinEffectiveLimitAfterDistribute	Recent aggregate outflow remains within effectiveLimit after distribute and stewardSpend (bounded-loop deferred property). Specific focus: recent outflow within effective limit after distribute.	✓	✓	
TG-17	HL_recentOutflowWithinEffectiveLimitAfterStewardSpend	recent outflow within effective limit after stewardSpend.	✓	✓	
TG-17	VT_recentOutflowWithinEffectiveLimitAlways	recent outflow within effective limit always.	✓	✓	
TG-18	VT_cumulativePerCallBounded	_checkAndRecordOutflow checks recentOutflow + amount <= effectiveLimit before append (bounded-loop deferred property)	✓	✓	
TG-19	ST_stewardSpendSuccessIsDoubleBounded	Successful stewardSpend is double-bounded by both aggregate outflow limit and steward budget, and appends one aggregate outflow record and one steward-spend record	✓	✓	
TG-21	ST_stewardSpendRemainingNonNegative	stewardSpend cannot underflow remaining budget: recentStewardSpend + amount <= limit	✓	✓	
TG-22	VT_distributeNeverConsumesStewardBudget	distribute never consumes steward spend history or changes steward budget parameters	✓	✓	
TG-23	VT_windDownTokenTransferDoesNotConsumeOutflowBudget	Wind-down transferTo and transferETHTo bypass aggregate outflow and steward-budget accounting. Specific focus: wind-down token transfer does not consume outflow budget.	✓	✓	
TG-23	VT_transferETHToExactDeltaAndBypassesAccounting	transfer ETH to exact delta and bypasses accounting.	✓	✓	
TG-24	ST_zeroAmountDistributeStillRecordsOutflowIfOtherwiseValid	Zero-amount distribute, if otherwise valid, records one outflow entry	✓	✓	
TG-25	UT_floorAllowsOutflow	Positive distribute within the floorAbsolute allowance is reachable with an empty outflow window	✓	✓	

Access Control / Validation (16 properties)

Specification: [specs/Governance/TreasuryGov.spec](#)

Rules passed: 16 Passed | Certora run: [TreasuryGov.conf](#)

Mitigation run: [TreasuryGov.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-26	ST_distributeOnlyOwner	distribute and stewardSpend succeed only for owner. Specific focus: distribute owner-only.	✓	✓	
TG-26	ST_stewardSpendOnlyOwner	stewardSpend owner-only.	✓	✓	
TG-27	ST_transferToOnlyWindDown	transferTo and transferETHTo succeed only for windDownContract. Specific focus: transferTo only wind-down.	✓	✓	
TG-27	ST_transferETHToOnlyWindDown	transfer ETH to only wind-down.	✓	✓	
TG-28	ST_zeroRecipientsRevert	distribute, stewardSpend, transferTo, and transferETHTo reject recipient == address(0)	✓	✓	
TG-29	ST_uninitializedTokenOutflowReverts	distribute and stewardSpend revert for uninitialized outflow tokens	✓	✓	
TG-30	ST_unauthorizedStewardTokenReverts	stewardSpend reverts for unauthorized steward-budget tokens	✓	✓	
TG-31	ST_zeroAmountStewardSpendReverts	stewardSpend rejects amount == 0; distribute intentionally does not	✓	✓	
TG-32	VS_ownerImmutable	owner is immutable	✓	✓	
TG-33	VT_outflowConfigChangesRequireOwnerOrActivation	Outflow config changes require owner, except permissionless activatePendingOutflowParams promotion	✓	✓	
TG-34	VT_stewardBudgetChangesRequireOwner	Steward budget changes require owner	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
TG-35	VT_windDownContractSetOneWay	windDownContractSet is one-way	✓	✓	
TG-36	ST_secondWindDownSetterReverts	A second setWindDownContract reverts	✓	✓	
TG-37	ST_setWindDownContractRejectsZero	setWindDownContract(address(0)) reverts	✓	✓	
TG-38	VT_windDownContractImmutableAfterSet	windDownContract is immutable after first set	✓	✓	
TG-39	ST_setWindDownContractOnlyOwner	setWindDownContract succeeds only for owner	✓	✓	

Concrete Value Transfers (3 properties)

Specification: [specs/Governance/TreasuryGovValueTransfer.spec](#)

Rules passed: 3 Passed | Certora run: [TreasuryGovValueTransfer.conf](#)

Mitigation run: [TreasuryGovValueTransfer.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-40	VT_distributeConcreteERC20BalanceDeltaExact	Concrete ERC20 distribute decreases treasury balance and increases external recipient balance by exactly amount, appends one aggregate outflow record whose amount equals the transfer amount, and does not append steward spend history	✓	✓	
TG-43	VT_stewardSpendConcreteERC20BalanceDeltaExact	Concrete ERC20 stewardSpend decreases treasury balance and increases external recipient balance by exactly amount, and appends one aggregate outflow record and one steward spend record, both with amount amount	✓	✓	
TG-45	VT_transferToConcreteERC20BalanceDeltaExactAndBypassesHistory	Concrete ERC20 wind-down transferTo decreases treasury balance and increases external recipient balance by exactly amount without appending outflow or steward history	✓	✓	

Steward Budgets (4 properties)

Specification: [specs/Governance/TreasuryGov.spec](#)

Rules passed: 4 Passed | Certora run: [TreasuryGov.conf](#)

Mitigation run: [TreasuryGov.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-47	HL_stewardBudgetViewHasNoCarryover	Steward budget views report zeros for unauthorized tokens and, for authorized tokens, report the configured limit with remaining = max(limit - spent, 0) and no carryover	✓	✓	
TG-49	ST_removeStewardBudgetTokenClearsBudgetP reservesHistory	removeStewardBudgetToken clears authorization, limit, and window while preserving steward spend history	✓	✓	
TG-50	ST_updateStewardBudgetTokenDoesNotErase History	updateStewardBudgetToken updates limit/window, preserves authorization, and does not erase history	✓	✓	
TG-51	HL_removeAddCyclePreservesStewardSpendHistory	Removing and re-adding a steward-budget token preserves spend history so recent spend still counts after reauthorization	✓	✓	

Lazy Activation / Parameter Changes (25 properties)

Specification: [specs/Governance/TreasuryGov.spec](#)

Rules passed: 25 Passed | Certora run: [TreasuryGov.conf](#)

Mitigation run: [TreasuryGov.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TG-52	HL_getEffectiveOutflowConfigMatchesHarnessEffectiveParams	getEffectiveOutflowConfig matches _effectiveParams and reports immutable floorAbsolute	✓	✓	
TG-53	HL_effectiveParamsMatchesLazyActivate	_effectiveParams matches the lazy-activation branch formula for absolute, bps, and window pending slots	✓	✓	
TG-54	ST_activatePromotesElapsedAbsolute	activatePendingOutflowParams promotes due absolute, bps, and window pending slots and clears their pending value/activation fields. Specific focus: activate promotes elapsed absolute.	✓	✓	
TG-54	ST_activatePromotesElapsedBps	activate promotes elapsed BPS.	✓	✓	
TG-54	ST_activatePromotesElapsedWindow	activate promotes elapsed window.	✓	✓	
TG-55	ST_bpsActivationDoesNotClearIndependentPendingSlots	Promoting one due pending slot does not clear unrelated not-yet-due pending slots	✓	✓	
TG-56	ST_scheduleAbsoluteUsesFullDelay	Absolute and bps loosening, and window shortenings, schedule a pending value at block.timestamp + LIMIT_ACTIVATION_DELAY and leave active params unchanged. Specific focus: schedule absolute uses full delay.	✓	✓	
TG-56	ST_scheduleBpsUsesFullDelay	schedule BPS uses full delay.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
TG-56	ST_scheduleWindowUsesFullDelay	schedule window uses full delay.	✓	✓	
TG-57	ST_tightenAbsoluteImmediate	Absolute and bps tightenings, and window lengthenings, apply immediately and clear matching pending slots. Specific focus: tighten absolute immediate.	✓	✓	
TG-57	ST_tightenBpsImmediate	tighten BPS immediate.	✓	✓	
TG-57	ST_tightenWindowImmediate	tighten window immediate.	✓	✓	
TG-58	ST_distributePromotesAllDuePendingBeforeEnforcement	distribute and stewardSpend promote all due pending slots before enforcement. Specific focus: distribute promotes all due pending before enforcement.	✓	✓	
TG-58	ST_stewardSpendPromotesAllDuePendingBeforeEnforcement	stewardSpend promotes all due pending before enforcement.	✓	✓	
TG-59	ST_dueBpsTighteningIsImmediate	When a due bps/absolute/window slot exists, a tighter replacement applies immediately and clears pending state. Specific focus: due BPS tightening is immediate.	✓	✓	
TG-59	ST_dueAbsoluteTighteningIsImmediate	due absolute tightening is immediate.	✓	✓	
TG-59	ST_dueWindowTighteningIsImmediate	due window tightening is immediate.	✓	✓	
TG-60	ST_dueBpsFurtherLooseningIsDelayed	When a due bps/absolute/window slot exists, a further loosening first activates the due value and schedules the new looser value with a full delay. Specific focus: due BPS further loosening is delayed.	✓	✓	
TG-60	ST_dueAbsoluteFurtherLooseningIsDelayed	due absolute further loosening is delayed.	✓	✓	
TG-60	ST_dueWindowFurtherLooseningIsDelayed	due window further loosening is delayed.	✓	✓	
TG-61	ST_absoluteLooseningSchedulesFutureActivation	Absolute, bps, and window loosening always write the pending value and activation timestamp instead of immediately loosening active limits. Specific focus: absolute loosening schedules future activation.	✓	✓	
TG-61	ST_bpsLooseningSchedulesFutureActivation	bPS loosening schedules future activation.	✓	✓	
TG-61	ST_windowLooseningSchedulesFutureActivation	window loosening schedules future activation.	✓	✓	
TG-62	HL_duePendingLooseningIsVisibleToExecutionTimeViews	Due pending loosening is visible to execution-time effective-limit views once activation has elapsed	✓	✓	
TG-63	HL_donationCanInflatePercentageLimit	Direct token donations can increase the percentage-driven component of effective limit when that component dominates	✓	✓	

Cross-Contract Rate Limit Composition (4 properties)

Specification: [specs/Governance/XRateLimit.spec](#)

Rules passed: 4 Passed | Certora run: [XRateLimit.conf](#)

Mitigation run: [XRateLimit.conf](#)

ID	Name	Description	Audit	Mitig	Notes
XRL-1	HL_rateLimitWindowBoundInductive	The rolling-window aggregate outflow for an inspected token is inductively preserved below effectiveLimit(token) outside immediate config setters	✓	✓	
XRL-2	VT_rateLimitOutflowAppendRespectsLimit	distribute and stewardSpend append paths preserve sumRecentOutflow(token) <= effectiveLimit(token)	✓	✓	
XRL-3	UT_rateLimitConstantsParityBetweenGovernorAndTreasury	Treasury LIMIT_ACTIVATION_DELAY matches the shared 24-day outflow activation delay constant	✓	✓	
XRL-4	ST_rateLimitSingleDistributeAboveLimitReverts	Without due pending activation, distribute(token, recipient, amount) reverts when amount exceeds remaining outflow budget	✓	✓	

TreasurySteward.sol

Steward Election / Lifecycle (19 properties)

Specification: [specs/Governance/TreasurySteward.spec](#)

Rules passed: 19 Passed | Certora run: [TreasurySteward.conf](#)

Mitigation run: [TreasurySteward.conf](#)

ID	Name	Description	Audit	Mitig	Notes
TS-1	UT_setupGhostsMatchGetters	CVL steward identity and termStart ghosts match the public getters	✓	✓	
TS-2	ST_electStewardOnlyTimelock	electSteward and removeSteward succeed only when msg.sender == timelock. Specific focus: elect steward timelock-only.	✓	✓	
TS-2	ST_removeStewardOnlyTimelock	remove steward timelock-only.	✓	✓	
TS-3	VT_stewardStateChangesRequireTimelock	Any change to steward identity or termStart requires a timelock caller	✓	✓	
TS-4	VS_timelockImmutable	timelock is immutable and non-zero. Specific focus: timelock immutable.	✓	✓	
TS-4	VS_timelockNonZero	timelock non zero.	✓	✓	
TS-5	ST_electStewardRejectsZero	electSteward(address(0)) reverts	✓	✓	
TS-6	ST_electStewardAtomic	electSteward(candidate) atomically sets currentSteward == candidate and termStart == block.timestamp	✓	✓	
TS-7	ST_removeStewardClearsIdentityAndTermStart	removeSteward clears currentSteward and preserves termStart	✓	✓	
TS-8	ST_termStartNonDecreasingOnElect	termStart is non-decreasing across successful elections when timestamps are monotonic	✓	✓	
TS-9	ST_electStewardAtomicReplace	A new electSteward(successor) immediately replaces the prior steward and resets termStart to now	✓	✓	
TS-10	HL_isStewardActiveEquivalence	isStewardActive $\Leftrightarrow$ currentSteward != address(0) $\wedge$ block.timestamp < termStart + TERM_DURATION	✓	✓	
TS-11	HL_isStewardActiveBoundaryBelow	isStewardActive is true at termEnd - 1 and false at termEnd. Specific focus: is steward active boundary below.	✓	✓	
TS-11	HL_isStewardActiveBoundaryAtEnd	is steward active boundary at end.	✓	✓	
TS-12	UT_termDurationIsConstant	TERM_DURATION == 180 days	✓	✓	
TS-13	HL_removedStewardIsInactiveAtAnyTimestamp	A removed or zero steward is inactive at every timestamp and termEnd == 0	✓	✓	
TS-14	ST_removeRevertIsAtomic	Reverted removeSteward calls leave currentSteward and termStart unchanged	✓	✓	
TS-15	UT_removeStewardAcceptsZeroIncumbent	removeSteward is allowed when the incumbent is already zero	✓	✓	
TS-16	VT_noNativeBalanceMutation	Non-payable TreasurySteward calls with zero msg.value cannot mutate the contract native balance	✓	✓	

ArmadaToken.sol

ERC20 Supply / Balances / Allowances (16 properties)

Specification: [specs/ArmadaToken/ERC20.spec](#), [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 16 Passed | Certora run: [ArmadaToken_extended.conf](#)

Mitigation run: [ArmadaToken_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AT-1	totalSupplyIsSumOfBalances	totalSupply == $\sum_a$ balanceOf(a)	✓	✓	
AT-2	zeroAddressNoBalance	balanceOf(address(0)) == 0	✓	✓	
AT-3	noChangeTotalSupply	Generic reachable functions do not change totalSupply. Specific focus: the generic total-supply-preservation check.	✓	✓	
AT-4	onlyAuthorizedCanTransfer	Only the holder or an approved spender can reduce an account balance	✓	✓	
AT-5	onlyHolderOfSpenderCanChangeAllowance	Allowance increases only through holder approve, holder increaseAllowance, or permit; allowance decreases only through spender transferFrom, holder allowance updates, or permit. Specific focus: only holder of spender can change allowance.	✓	✓	
AT-5	VT_onlyHolderOrSpenderCanChangeAllowance	only holder or spender can change allowance.	✓	✓	
AT-6	transfer	Armada-aware transfer updates sender/recipient balances exactly, preserves other balances, and reverts only for zero endpoints, insufficient balance, or transfer-gate failure. Specific focus: the transfer implementation path.	✓	✓	
AT-6	HL_transferBalanceSemantics	transfer balance semantics.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AT-7	transferFrom	Armada-aware transferFrom updates balances and finite allowance exactly, preserves other balances, and respects the transfer gate. Specific focus: the transferFrom implementation path.	✓	✓	
AT-7	HL_transferFromBalanceAndAllowanceSemantics	transferFrom balance and allowance semantics.	✓	✓	
AT-8	approve	approve sets exactly allowance(holder, spender) == amount and leaves unrelated allowances unchanged	✓	✓	
AT-9	permit	permit sets allowance, increments the owner's nonce by 1, respects deadline, and leaves unrelated nonces/allowances unchanged. Specific focus: the permit implementation path.	✓	✓	
AT-9	ST_permitRespectsDeadline	permit respects deadline.	✓	✓	
AT-10	HL_transferSelfTransferNoOp	Self transfer and self transferFrom are balance-preserving and total-supply-preserving no-ops. Specific focus: transfer self transfer no-op.	✓	✓	
AT-10	HL_transferFromSelfTransferNoOp	transferFrom self transfer no-op.	✓	✓	
AT-11	deploy_initialHolderNonZero	Constructor post-state preserves zero-address balance, implying the initial holder was non-zero in successful deployments	✓	✓	

Transferability Gate / Whitelist (14 properties)

Specification: [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 25 Passed | Certora run: [ArmadaToken_extended.conf](#)

Mitigation run: [ArmadaToken_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AT-12	ST_setTransferableOnlyAcceptsTrue	setTransferable(false) always reverts	✓	✓	
AT-13	ST_setTransferableTrueWhenAlreadyTrueReverts	setTransferable(true) reverts once transferable == true	✓	✓	
AT-14	UT_setTransferableLiveness	Authorized windDownContract or timelock callers can enable unrestricted transfers	✓	✓	
AT-15	VT_transferableChangeRequiresAuthorizedCaller	transferable changes only via authorized setTransferable and is monotonic once true. Specific focus: transferable change requires authorized caller.	✓	✓	
AT-15	VT_transferableMonotonic	transferable monotonic.	✓	✓	
AT-16	ST_unauthorizedSetTransferableBlocked	Unauthorized callers cannot enable unrestricted transfers	✓	✓	
AT-17	ST_nonWhitelistedTransferBlockedWhileRestricted	While restricted, non-whitelisted senders cannot transfer	✓	✓	
AT-18	ST_transferFromGateRestrictsNonWhitelisted	While restricted, transferFrom success implies transferWhitelist[from]	✓	✓	
AT-19	HL_transferGateDoesNotBlockReceipts	While restricted, recipient whitelist status is not required when the sender is whitelisted	✓	✓	
AT-20	HL_transferGateBypassOnlyForMints	Only mint-like from == address(0) paths could bypass the transfer gate; no reachable mint/burn path changes supply. Specific focus: transfer gate bypass only for mints.	✓	✓	
AT-21	VT_whitelistAddOnlyNonDeployer	Non-deployer whitelist entries are add-only	✓	✓	
AT-22	VT_removeDeployerFromWhitelistScopedToOwnEntry	removeDeployerFromWhitelist only clears the deployer's own entry and does not affect other whitelist entries	✓	✓	
AT-23	VT_whitelistWritesGated	Whitelist writes occur only through initWhitelist by deployer, removeDeployerFromWhitelist by deployer, or addToWhitelist by timelock	✓	✓	
AT-24	ST_addToWhitelistOnlyTimelock	Non-timelock callers cannot call addToWhitelist	✓	✓	

One-Shot Setup / Access Control (25 properties)

Specification: [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 25 Passed | Certora run: [ArmadaToken_extended.conf](#)
Mitigation run: [ArmadaToken_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AT-25	ST_initWhitelistRejectsZeroAddress	initWhitelist, initNoDelegation, and initAuthorizedDelegators reject arrays containing address(0). Specific focus: init whitelist rejects zero address.	✓	✓	
AT-25	ST_initNoDelegationRejectsZeroAddress	init no delegation rejects zero address.	✓	✓	
AT-25	ST_initAuthorizedDelegatorsRejectsZeroAddress	init authorized delegators rejects zero address.	✓	✓	
AT-26	ST_setWindDownContractRejectsZeroAddress	setWindDownContract(address(0)), addToWhitelist(address(0)), and addAuthorizedDelegator(address(0)) revert. Specific focus: set wind-down contract rejects zero address.	✓	✓	
AT-26	ST_addToWhitelistRejectsZeroAddress	add to whitelist rejects zero address.	✓	✓	
AT-26	ST_addAuthorizedDelegatorRejectsZeroAddress	add authorized delegator rejects zero address.	✓	✓	
AT-27	ST_initWhitelistSecondCallReverts	initWhitelist, setWindDownContract, initNoDelegation, and initAuthorizedDelegators are one-shot and revert on second call. Specific focus: init whitelist second call reverts.	✓	✓	
AT-27	ST_setWindDownContractSecondCallReverts	set wind-down contract second call reverts.	✓	✓	
AT-27	ST_initNoDelegationSecondCallReverts	init no delegation second call reverts.	✓	✓	
AT-27	ST_initAuthorizedDelegatorsSecondCallReverts	init authorized delegators second call reverts.	✓	✓	
AT-28	ST_initWhitelistOnlyDeployer	Initial setup functions succeed only for tokenDeployer. Specific focus: init whitelist only deployer.	✓	✓	
AT-28	ST_setWindDownContractOnlyDeployer	set wind-down contract only deployer.	✓	✓	
AT-28	ST_initNoDelegationOnlyDeployer	init no delegation only deployer.	✓	✓	
AT-28	ST_initAuthorizedDelegatorsOnlyDeployer	init authorized delegators only deployer.	✓	✓	
AT-29	ST_addAuthorizedDelegatorOnlyTimelock	addAuthorizedDelegator succeeds only for timelock	✓	✓	
AT-30	ST_removeDeployerFromWhitelistOnlyDeployer	removeDeployerFromWhitelist succeeds only for tokenDeployer	✓	✓	
AT-31	VT_whitelistInitializedOneWay	whitelistInitialized, windDownContractSet, noDelegationSet, and authorizedDelegatorsInitialized are one-way flags. Specific focus: whitelist initialized one way.	✓	✓	
AT-31	VT_windDownContractSetOneWay	wind-down contract set one way.	✓	✓	
AT-31	VT_noDelegationSetOneWay	no delegation set one way.	✓	✓	
AT-31	VT_authorizedDelegatorsInitializedOneWay	authorized delegators initialized one way.	✓	✓	
AT-32	VT_initFlagsOnlySetByDedicatedInit	Init flags change only through their dedicated init/set functions. Specific focus: init flags only set by dedicated init.	✓	✓	
AT-32	deploy_initFlagsOnlySetByInitFunctions	the deployment-time check for init flags only set by init functions.	✓	✓	
AT-33	VT_windDownContractImmutableAfterSet	windDownContract is immutable after first set	✓	✓	
AT-34	VS_timelockImmutable	timelock and tokenDeployer are immutable constructor references. Specific focus: timelock immutable.	✓	✓	
AT-34	VS_tokenDeployerImmutable	token deployer immutable.	✓	✓	

Delegation / Voting Authority (16 properties)

Specification: [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 16 Passed | Certora run: [ArmadaToken_extended.conf](#)

Mitigation run: [ArmadaToken_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AT-35	ST_delegateRevertsForNoDelegation	delegate reverts for callers in noDelegation	✓	✓	
AT-36	ST_noDelegationBlocksDelegateOnBehalf	delegateOnBehalf reverts when the delegator is in noDelegation, even for authorized callers. Specific focus: no delegation blocks delegateOnBehalf.	✓	✓	
AT-36	ST_noDelegationBlocksDelegate	no delegation blocks delegate.	✓	✓	
AT-37	HL_noDelegationAddressesKeepNoOwnDelegate	A no-delegation address cannot acquire its own delegate entry	✓	✓	
AT-38	VT_noDelegationImmutableAfterInit	noDelegation entries are immutable after initNoDelegation	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AT-39	VT_authorizedDelegatorAddOnly	authorizedDelegator entries are add-only and cannot be removed	✓	✓	
AT-40	VT_authorizedDelegatorWritesGated	Authorized-delegator writes occur only through deployer initialization or timelock add path	✓	✓	
AT-41	ST_unauthorizedDelegateOnBehalfBlocked	Unauthorized callers cannot use delegateOnBehalf or transferAndDelegate. Specific focus: unauthorized delegateOnBehalf blocked.	✓	✓	
AT-41	ST_unauthorizedTransferAndDelegateBlocked	unauthorized transferAndDelegate blocked.	✓	✓	
AT-42	VT_delegatesMutateOnlyViaDelegationFns	delegates(a) mutates only through delegate, delegateBySig, or delegateOnBehalf	✓	✓	
AT-43	VT_delegationPreservesBalances	Delegation functions preserve balances and total supply	✓	✓	
AT-44	VT_transferPreservesDelegates	Transfer functions preserve delegation choices	✓	✓	
AT-45	UT_delegationLiveness	Non-noDelegation addresses can delegate successfully. Specific focus: delegation liveness.	✓	✓	
AT-46	UT_delegateOnBehalfLiveness	Authorized delegators can delegateOnBehalf for non-blocked delegators. Specific focus: delegateOnBehalf liveness.	✓	✓	
AT-46	HL_authorizedDelegateOnBehalfStillRespectsNoDelegation	authorized delegateOnBehalf still respects no delegation.	✓	✓	
AT-47	ST_delegateBySigRespectsDeadline	delegateBySig respects expiry	✓	✓	

Cross-Contract Locked Holder Voting (6 properties)

Specification: [specs/Governance/XLockedNoVotes.spec](#), [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 6 Passed | Certora run: [XLockedNoVotes.conf](#)

Mitigation run: [XLockedNoVotes.conf](#)

ID	Name	Description	Audit	Mitig	Notes
XNV-1	VT_lockedNoDelegationImmutableAfterInit	noDelegation[a] entries can change only through initNoDelegation	✓	✓	
XNV-2	HL_lockedAddressesHaveZeroVotes	A locked noDelegation address with zero votes and no inbound delegation keeps zero votes across non-delegation entypoints	✓	✓	
XNV-3	ST_lockedNoDelegationBlocksDelegateOnBehalf	delegateOnBehalf(a, delegatee) and transferAndDelegate(_, a, _, delegatee) revert when a is in the noDelegation set. Specific focus: locked no delegation blocks delegateOnBehalf.	✓	✓	
XNV-3	ST_lockedNoDelegationBlocksTransferAndDelegate	locked no delegation blocks transferAndDelegate.	✓	✓	
XNV-3	HL_authorizedTransferAndDelegateStillRespectsNoDelegation	authorized transferAndDelegate still respects no delegation.	✓	✓	
XNV-4	UT_lockedNoDelegationGhostConsistency	The noDelegation public getter stays synchronized with the CVL ghost mirror	✓	✓	

Validation / Offensive Checks (19 properties)

Specification: [specs/ArmadaToken/ArmadaToken.spec](#)

Rules passed: 19 Passed | Certora run: [ArmadaToken_extended.conf](#)

Mitigation run: [ArmadaToken_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AT-48	anti_delegateOnBehalfBypassesNoDelegation	Anti-check: delegateOnBehalf cannot bypass noDelegation	✓	✓	
AT-49	anti_unauthorizedSetTransferable	Anti-check: unauthorized callers cannot enable transfers	✓	✓	
AT-50	anti_transferGateReversal	Anti-check: transferable cannot reverse from true to false	✓	✓	
AT-51	anti_unauthorizedDelegateOnBehalf	Anti-check: unauthorized callers cannot call delegateOnBehalf successfully	✓	✓	
AT-52	anti_nonWhitelistedTransferWhileRestricted	Anti-check: non-whitelisted senders cannot transfer while restricted	✓	✓	
AT-53	anti_authorizedDelegatorRemoval	Anti-check: authorized delegator entries cannot be removed	✓	✓	
AT-54	anti_totalSupplyMutation	Anti-check: total supply cannot mutate	✓	✓	
AT-55	anti_whitelistRemovalNonDeployer	Anti-check: non-deployer whitelist entries cannot be removed	✓	✓	
AT-56	anti_governanceByNonTimelock	Anti-check: non-timelock callers cannot use governance add paths. Specific focus: the counterexample check for governance by non timelock.	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
AT-56	ST_governanceByNonTimelockBlocked	governance by non timelock blocked.	✓	✓	
AT-57	UT_transferLivenessUnrestricted	Validation: unrestricted transfers are reachable for positive, sufficiently funded holders. Specific focus: transfer liveness unrestricted.	✓	✓	
AT-57	val_transferLivenessUnrestricted	the validation witness for transfer liveness unrestricted.	✓	✓	
AT-58	UT_whitelistedTransferLivenessRestricted	Validation: whitelisted transfers are reachable while restricted. Specific focus: whitelisted transfer liveness restricted.	✓	✓	
AT-58	val_whitelistedTransferLivenessRestricted	the validation witness for whitelisted transfer liveness restricted.	✓	✓	
AT-59	UT_transferAndDelegateLiveness	Validation: delegation, authorized delegateOnBehalf, and authorized transferAndDelegate are reachable for valid non-blocked accounts. Specific focus: transferAndDelegate liveness.	✓	✓	
AT-59	val_delegationLiveness	the validation witness for delegation liveness.	✓	✓	
AT-59	val_delegateOnBehalfLiveness	the validation witness for delegateOnBehalf liveness.	✓	✓	
AT-59	val_transferAndDelegateLiveness	the validation witness for transferAndDelegate liveness.	✓	✓	
AT-60	val_setTransferableLiveness	Validation: authorized setTransferable(true) is reachable and sets the gate true. Specific focus: the validation witness for setTransferable liveness.	✓	✓	

Winddown / Redemption

ArmadaWindDown.sol (34 properties)

Specification: [specs/WindDown/ArmadaWindDown.spec](#)

Rules passed: 33 Passed, 1 Violated | **Certora run:** [ArmadaWindDown_extended.conf](#)

Mitigation run: All Passed After Mitig. [ArmadaWindDown_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
WD-1	VS_triggeredIffTriggerTime	triggered $\Leftrightarrow$ triggerTime > 0	✓	✓	
WD-2	VS_revenueThresholdPositivePreTrigger	Pre-trigger revenueThreshold > 0 and windDownDeadline > 0. Specific focus: revenue threshold positive pre trigger.	✓	✓	
WD-2	VS_deadlinePositivePreTrigger	deadline positive pre trigger.	✓	✓	
WD-3	VT_triggeredMonotonic	triggered is monotonic and never resets	✓	✓	
WD-4	VT_triggerTimeWriteOnce	triggerTime is written exactly on the trigger transition as block.timestamp and is immutable after trigger	✓	✓	
WD-5	ST_triggerWindDownNoDoubleTrigger	triggerWindDown and governanceTriggerWindDown cannot succeed twice. Specific focus: trigger wind-down no double trigger.	✓	✓	
WD-5	ST_governanceTriggerNoDoubleTrigger	governance trigger no double trigger.	✓	✓	
WD-6	ST_triggerWindDownSuccessImpliesPreconditions	Permissionless triggerWindDown success implies pre-trigger state, block.timestamp > windDownDeadline, and stored revenue below threshold	✓	✓	
WD-7	ST_permissionlessTriggerFailsWhenRevenueMeetsThreshold	Permissionless trigger reverts when stored revenue meets or exceeds threshold	✓	✓	
WD-8	ST_permissionlessTriggerFailsWhenSyncedRevenueMeetsThreshold	Permissionless trigger reverts when synced pending stablecoin revenue would meet or exceed threshold	X	✓	See #97 Passed After Fix: 09e4016
WD-9	ST_permissionlessTriggerFailsBeforeDeadline	Permissionless trigger reverts at or before the wind-down deadline	✓	✓	
WD-10	UT_permissionlessTriggerSucceedsWhenGateSatisfied	Permissionless trigger is reachable once deadline passed and revenue is below threshold	✓	✓	
WD-11	ST_governanceTriggerSuccessImpliesPreconditions	Governance trigger succeeds for timelock callers pre-trigger and stamps trigger state	✓	✓	
WD-12	UT_governanceTriggerSucceedsForTimelock	Timelock governance trigger is reachable pre-trigger	✓	✓	
WD-13	VT_permissionlessTriggerSetsLocalState	Both trigger paths keep triggered and triggerTime synchronized atomically. Specific focus: permissionless trigger sets local state.	✓	✓	
WD-13	VT_governanceTriggerSetsLocalState	governance trigger sets local state.	✓	✓	
WD-14	ST_setRevenueThresholdAuth	setRevenueThreshold succeeds only for timelock pre-trigger with nonzero threshold	✓	✓	
WD-15	ST_setWindDownDeadlineAuth	setWindDownDeadline succeeds only for timelock pre-trigger with a future deadline	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
WD-16	VT_revenueThresholdImmutablePostTrigger	After trigger, revenueThreshold and windDownDeadline are immutable and their setters revert. Specific focus: revenue threshold immutable post trigger.	✓	✓	
WD-16	VT_windDownDeadlineImmutablePostTrigger	wind-down deadline immutable post trigger.	✓	✓	
WD-16	ST_setRevenueThresholdRevertsPostTrigger	set revenue threshold reverts post trigger.	✓	✓	
WD-16	ST_setWindDownDeadlineRevertsPostTrigger	set wind-down deadline reverts post trigger.	✓	✓	
WD-17	VT_revenueThresholdOnlyChangesViaSetter	revenueThreshold changes only through setRevenueThreshold; windDownDeadline changes only through setWindDownDeadline. Specific focus: revenue threshold only changes via setter.	✓	✓	
WD-17	VT_windDownDeadlineOnlyChangesViaSetter	wind-down deadline only changes via setter.	✓	✓	
WD-18	VT_deadlineCannotBeZero	No reachable path can zero windDownDeadline	✓	✓	
WD-19	VT_permissionlessTriggerDoesNotChangeParameters	Triggering wind-down does not change revenueThreshold or windDownDeadline. Specific focus: permissionless trigger does not change parameters.	✓	✓	
WD-19	VT_governanceTriggerDoesNotChangeParameters	governance trigger does not change parameters.	✓	✓	
WD-20	ST_sweepTokenSuccessImpliesPreconditions	sweepToken(token) preserves trigger state and is reachable post-trigger for non-ARM tokens with positive treasury balance. Specific focus: sweep token success implies preconditions.	✓	✓	
WD-20	UT_sweepTokenPermissionlessPostTrigger	sweep token permissionless post trigger.	✓	✓	
WD-21	ST_sweepEthSuccessImpliesPreconditions	sweepETH preserves trigger state and is reachable post-trigger with positive treasury ETH	✓	✓	
WD-22	ST_armCanNeverBeSwept	sweepToken(armToken) always reverts	✓	✓	
WD-23	HL_sweepTokenDrainsTreasuryToRedemption	Token sweep drains the full treasury token balance to redemption when treasury and redemption are distinct	✓	✓	
WD-24	VS_immutablesNeverChange	armToken, treasury, governor, redemptionContract, pauseContract, revenueCounter, and timelock are immutable	✓	✓	
WD-25	UT_windDownTriggeredEventOnPermissionlessTrigger	Successful permissionless trigger reaches the event/emit path and sets local triggered state	✓	✓	

Cross-Contract Wind-Down Atomicity (9 properties)

Specification: [specs/WindDown/XWindDownAtomic.spec](#)

Rules passed: 9 Passed | Certora run: [XWindDownAtomic.conf](#)

Mitigation run: [XWindDownAtomic.conf](#)

ID	Name	Description	Audit	Mitig	Notes
XWD-1	HL_windDownAtomicTriggerAllOrNothing	After any ArmadaWindDown call, triggered() implies ARM transferability, Governor wind-down activation, and ShieldPauseController wind-down activation	✓	✓	
XWD-2	VT_windDownNoExternalFlagRollback	External wind-down ghost flags for token, Governor, and pause controller are one-way once set	✓	✓	
XWD-3	HL_windDownPermissionlessTriggerSetsTokenTransferable	Permissionless triggerWindDown makes ARM transferable when it triggers	✓	✓	
XWD-4	HL_windDownGovernanceTriggerSetsTokenTransferable	Governance governanceTriggerWindDown makes ARM transferable when it triggers	✓	✓	
XWD-5	HL_windDownPermissionlessTriggerActivatesGovernor	Permissionless triggerWindDown activates Governor wind-down when it triggers	✓	✓	
XWD-6	HL_windDownGovernanceTriggerActivatesGovernor	Governance governanceTriggerWindDown activates Governor wind-down when it triggers	✓	✓	
XWD-7	HL_windDownPermissionlessTriggerActivatesPauseController	Permissionless triggerWindDown activates ShieldPauseController wind-down when it triggers	✓	✓	
XWD-8	HL_windDownGovernanceTriggerActivatesPauseController	Governance governanceTriggerWindDown activates ShieldPauseController wind-down when it triggers	✓	✓	
XWD-9	UT_windDownTriggerFlipsAllThreeExternals	Governance triggering can reach a coherent post-trigger state with all external wind-down flags set	✓	✓	

ArmadaRedemption.sol (22 properties)

Specification: [specs/WindDown/ArmadaRedemption.spec](#)

Rules passed: 21 Passed, 1 Violated | Certora run: [ArmadaRedemption_extended.conf](#)
Mitigation run: All Passed After Mitig. [ArmadaRedemption_extended.conf](#)

ID	Name	Description	Audit	Mitig	Notes
AR-1	VS_circulatingSupplyNoUnderflow	Circulating-supply subtraction cannot underflow when excluded addresses are distinct	✓	✓	
AR-2	ST_setWindDownOnlyAdmin	Only immutable admin may wire windDown, and only once	✓	✓	
AR-3	ST_setWindDownOneShot	Once non-zero, windDown never changes. Specific focus: set wind-down one shot.	✓	✓	
AR-3	VT_windDownMonotonic	wind-down monotonic.	✓	✓	
AR-4	ST_redeemRevertsOnZeroArmAmount	redeem(0, _, _) always reverts	✓	✓	
AR-5	ST_redeemRevertsWhenNotTransferable	redeem reverts while ARM transfers are disabled	✓	✓	
AR-6	ST_redeemSuccessImpliesBasicPreconditions	Any broken basic gate forces redeem to revert: zero ARM, non-transferable ARM, unwired wind-down, no trigger time, redemption delay not elapsed, or zero circulating supply	✓	✓	
AR-7	ST_redeemRevertsBeforeDelayBoundary	redeem reverts one second before triggerTime + REDEMPTION_DELAY	✓	✓	
AR-8	ST_redeemRevertsWhenNoPayout	redeem reverts when all requested ERC20 and ETH payouts would be zero	✓	✓	
AR-9	ST_redeemRejectsUnsortedTokens	tokens[] must be strictly ascending; unsorted or duplicate arrays revert	✓	✓	
AR-10	ST_redeemRejectsArmInTokens	tokens[] cannot include armToken	✓	✓	
AR-11	UT_redeemSucceedsWhenConditionsMet	Redemption is reachable on a valid post-trigger, transferable, delayed path with positive payout	✓	✓	
AR-12	VT_circulatingSupplyDecreasesByArmAmount	Non-excluded redemption reduces circulatingSupply by exactly armAmount	✓	✓	
AR-13	VT_circulatingSupplyUnchangedForExcludedRedeemer	Redemption from an excluded address leaves circulatingSupply unchanged	✓	✓	
AR-14	VT_armBalanceMonotonic	Redemption contract ARM balance is monotonic non-decreasing	✓	✓	
AR-15	HL_perTokenPayoutMatchesFloorProRata	Requested ERC20 payout equals floor(balanceOf(redemption, token) × armAmount / circulatingSupply)	✓	✓	
AR-16	HL_ethPayoutMatchesFloorProRata	Requested ETH payout uses the same floor pro-rata formula	✓	✓	
AR-17	VS_nonArmValueLeavesOnlyViaRedeem	Non-ARM token balances of redemption can decrease only through redeem	✓	✓	
AR-18	VS_immutableNeverChange	admin, armToken, treasury, revenueLock, crowdfundContract, and REDEMPTION_DELAY are immutable	✓	✓	
AR-19	ST_redeemAtomicOnRevert	Guaranteed reverting redemption leaves ARM, requested token, and ETH balances unchanged	✓	✓	
AR-20	UT_successfulRedeemLocksExactArmAmount	Successful redemption locks exactly armAmount ARM in the redemption contract and debits that amount from the redeemer	✓	✓	
AR-21	UT_redeemSelectedPayoutPathsWellFormed	A token-only redemption path is rejected when both the selected token payout and the omitted ETH payout would be positive, preventing partial payout selection	X	✓	See FV-01 Fixed in commit: cf1ae6fc

Revenue

RevenueCounter.sol (19 properties)

Specification: [specs/Revenue/RevenueCounter.spec](#)

Rules passed: 19 Passed | Certora run: [RevenueCounter.conf](#)

Mitigation run: [RevenueCounter.conf](#)

ID	Name	Description	Audit	Mitig	Notes
RC-1	VS_initializerWriteCount	Proxy initialization writes occur at most once; initialized mirror and write count stay consistent	✓	✓	
RC-2	VT_recognizedRevenueMonotonic	recognizedRevenueUsd is monotonic non-decreasing	✓	✓	
RC-3	VT_syncDeltaNonNegative	syncStablecoinRevenue cannot apply a negative delta when lastSyncedCumulative <= cumulativeFeesCollected	✓	✓	
RC-4	ST_attestRevenueOnlyOwner	addRevenue and attestRevenue succeed only for owner. Specific focus: attest revenue owner-only.	✓	✓	
RC-4	ST_addRevenueOnlyOwner	add revenue owner-only.	✓	✓	
RC-5	ST_setFeeCollectorOnlyOwner	setFeeCollector succeeds only for owner	✓	✓	
RC-6	VT_revenueAllowlist	recognizedRevenueUsd mutates only through	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
		syncStablecoinRevenue, addRevenue, attestRevenue, setFeeCollector, or initialization			
RC-7	VT_ownerAllowlist	owner mutates only through transferOwnership, renounceOwnership, or initialization	✓	✓	
RC-8	ST_syncRevertsWithoutFeeCollector	syncStablecoinRevenue reverts when feeCollector == address(0)	✓	✓	
RC-9	ST_attestRevertsWhenNotMonotonic	attestRevenue(v) reverts when v < recognizedRevenueUsd	✓	✓	
RC-10	ST_initializeNotCallableTwice	initialize is callable at most once	✓	✓	
RC-11	VT_syncUpdatesLastSyncedOnNonZeroDelta	If syncStablecoinRevenue applies nonzero revenue, lastSyncedCumulative updates to the reported cumulative fee value	✓	✓	
RC-12	HL_setFeeCollectorDrainsOldCollector	setFeeCollector drains pending revenue from the old collector before switching	✓	✓	
RC-13	VT_setFeeCollectorResetsBaseline	After setFeeCollector(new), baseline is 0 for new == address(0) and cumulativeFeesCollected(new) otherwise	✓	✓	
RC-14	ST_upgradeToOnlyOwner	upgradeTo and upgradeToAndCall succeed only for the owner. Specific focus: upgrade to owner-only.	✓	✓	
RC-14	ST_upgradeToAndCallOnlyOwner	upgrade to and call owner-only.	✓	✓	
RC-15	UT_attestRevenueLiveness	Valid owner attestation and valid sync are reachable. Specific focus: attest revenue liveness.	✓	✓	
RC-15	UT_syncLiveness	sync liveness.	✓	✓	
RC-16	VT_addRevenueIncrementsExactly	Successful addRevenue(deltaUsd) increases recognizedRevenueUsd by exactly deltaUsd	✓	✓	

RevenueLock.sol (40 properties)

Specification: [specs/Revenue/RevenueLock.spec](#)

Rules passed: 38 Passed, 2 Violated | Certora run: [RevenueLock.conf](#)

Mitigation run: All Passed After Mitig. [RevenueLock.conf](#)

ID	Name	Description	Audit	Mitig	Notes
RL-1	VS_solveny	Once activated, lock solvency holds: $\text{arm.balanceOf}(\text{revenueLock}) + \Sigma \text{ released} \geq \text{totalAllocation}$	X	✓	See FV-02 Passed After Fix: c05d5f
RL-2	HL_allocationSumMatchesTotal	$\Sigma \text{ allocation} == \text{totalAllocation}$	✓	✓	
RL-3	VS_totalReleasedBounded	$\Sigma \text{ released} \leq \text{totalAllocation}$	✓	✓	
RL-4	VT_releaseCorrectness	After activation, release(delegatee) transfers exactly releasable(msg.sender), updates released[msg.sender], and reaches current entitlement	✓	✓	
RL-5	VT_syncPreservesNonRatchetState	syncObservedRevenue preserves allocations, released amounts, total allocation, max rate, beneficiary count, revenue counter, ARM token, and does not transfer ARM; after freeze it is a full no-op including maxObservedRevenue and lastSyncTimestamp. Specific focus: sync preserves non ratchet state.	✓	✓	
RL-5	VT_frozenSyncNoOps	frozen sync no-ops.	✓	✓	
RL-6	VT_syncRatchetAdvancement	Live sync advances maxObservedRevenue monotonically, bounded by elapsed-time rate limit and reported revenue, and sets lastSyncTimestamp == block.timestamp	✓	✓	
RL-7	UT_ratchetExactAdvanceOnSync	In live mode, when rate budget permits full catch-up, sync sets maxObservedRevenue == currentRevenue()	✓	✓	
RL-8	VT_maxObservedRevenueMonotonic	maxObservedRevenue is monotonic across all calls	✓	✓	
RL-9	VT_ratchetRateBounded	Ratchet advance across any call is bounded by $(\text{elapsed} \times \text{MAX_REVENUE_INCREASE_PER_DAY}) / 1 \text{ day}$	✓	✓	
RL-10	VT_lastSyncMonotonic	lastSyncTimestamp is monotonic and never exceeds block.timestamp. Specific focus: last sync monotonic.	✓	✓	
RL-10	VS_lastSyncBoundedByNow	last sync bounded by now.	✓	✓	
RL-11	VT_lastSyncUpdatedOnRelease	Live successful release sets lastSyncTimestamp == block.timestamp; frozen release preserves the frozen timestamp	✓	✓	
RL-12	HL_releasableViewConsistency	$\text{releasable}(u) == \max(\text{allocation}[u] \times \text{unlockPercentage} / 10000 - \text{released}[u], 0)$	✓	✓	
RL-13	UT_releaseNoSideEffectOnOthers	Release by one beneficiary does not change another beneficiary's released or allocation	✓	✓	
RL-14	VT_counterReadOnlyThroughRatchet	Only counter-reading methods may advance the ratchet to the	✓	✓	

ID	Name	Description	Audit	Mitig	Notes
		capped observed revenue preview; this includes freezeAtWindDown's final pre-freeze ratchet update			
RL-15	VT_revenueCounterImmutable	revenueCounter, allocation[u], and beneficiaryCount are immutable post-construction. Specific focus: revenue counter immutable.	✓	✓	
RL-15	VT_allocationImmutable	allocation immutable.	✓	✓	
RL-15	VT_beneficiaryCountImmutable	beneficiary count immutable.	✓	✓	
RL-16	VT_releasedMonotonic	released[u] is monotonic non-decreasing	✓	✓	
RL-17	VS_releasedBoundedByEntitlement	released[u] <= allocation[u] × unlockPercentage / 10000	✓	✓	
RL-18	VT_releaseCallsDelegate	Successful release(delegatee) calls armToken.transferAndDelegate(msg.sender, amount, delegatee) with exact recipient/delegatee	✓	✓	
RL-19	ST_releaseRequiresBeneficiary	release reverts for non-beneficiaries	✓	✓	
RL-20	ST_releaseRevertsOnZeroReleasable	release reverts when releasable amount is zero	✓	✓	
RL-21	ST_releaseRejectsZeroDelegatee	release(address(0)) reverts	✓	✓	
RL-22	VT_balanceDeltaMatchesReleasedDelta	For real external callers, any decrease in RevenueLock ARM balance equals the increase in aggregate released amount	✓	✓	
RL-23	HL_noExtractionByNonBeneficiary	Non-beneficiaries cannot receive ARM from RevenueLock through any call	✓	✓	
RL-24	HL_cannotExceedAllocation	Beneficiary released[u] can never exceed allocation[u]	✓	✓	
RL-25	HL_compromisedCounterBypassRateLimit	In live mode, a compromised revenue counter reporting above the rate budget cannot bypass the ratchet cap	✓	✓	
RL-26	VS_lastSyncNeverZero	lastSyncTimestamp cannot become zero after deployment once positive	✓	✓	
RL-27	VT_ratchetCannotDecreaseAdversarial	Ratchet cannot decrease when the counter reports a lower value than the current ratchet	✓	✓	
RL-28	ST_doubleClaimProtection	A second same-block release reverts and leaves released[u] equal to exact entitlement	✓	✓	
RL-29	HL_multiSyncRateLimit	In live mode, splitting time across multiple syncs cannot advance maxObservedRevenue beyond a one-shot sync over the same elapsed time	✓	✓	
RL-30	HL_noSweepNoBackdoor	Only release can decrease RevenueLock ARM balance; non-release calls cannot sweep ARM	✓	✓	
RL-31	ST_releaseOnlyToCaller	release sends ARM only to the caller-beneficiary, never to third parties	✓	✓	
RL-32	UT_releaseLiveness	Positive release, sync, live ratchet advancement, and partial milestone unlock states are reachable. Specific focus: release liveness.	✓	✓	
RL-32	UT_syncLiveness	sync liveness.	✓	✓	
RL-32	UT_ratchetLiveness	ratchet liveness.	✓	✓	
RL-32	UT_milestoneLiveness	milestone liveness.	✓	✓	
RL-33	UT_releasePreviewMatchesAvailableBacking	release rejects a positive preview when the contract can cover that beneficiary preview but cannot fully back the total outstanding unreleased allocation	✗	✓	See FV-02 Passed After Fix: c05d5f

Findings and Real Issues

ID	Severity	Issue	Status
FV-01	Low	ArmadaRedemption redeem can forfeit the caller's pro-rata ETH share	Fixed
FV-02	Info	RevenueLock claims are order-dependent if the contract is underfunded	Fixed

FV-01 [Low] ArmadaRedemption redeem can forfeit the caller's pro-rata ETH share (#81)

ArmadaRedemption.redeem allows a caller to opt out of receiving native ETH by passing includeETH = false, while still transferring their ARM into the redemption contract and receiving ERC20 payouts.

As a result, a caller can unintentionally forfeit their pro-rata ETH share. The unclaimed ETH remains in the redemption contract and can be captured by subsequent redeemers.

From an economic perspective, if a caller can receive native ETH successfully, choosing includeETH = true is strictly better than choosing includeETH = false because the caller receives more total value in the same redemption flow. Opting out of ETH therefore does not provide a financial benefit to the redeemer and instead leaves part of their pro-rata entitlement behind for others to claim later.

X Violated here: <https://prover.certora.com/output/727987/fbe7bceb6fc346a69a642422be7874cd/>

✓ Passed After Fix: <https://prover.certora.com/output/727987/aaf14c0fb8974f93b74a6a7da9ee5c85/>

FV-02 [Info] RevenueLock claims are order-dependent if the contract is underfunded (#77)

RevenueLock stores beneficiary allocations and totalAllocation, but it does not enforce an on-chain funding check before claims are used.

If the contract holds less than the intended 2,400,000 ARM, beneficiaries can still appear entitled, but actual payouts depend on the remaining token balance in the contract.

1. Early beneficiaries may claim successfully.
2. Later beneficiaries may revert once balance is insufficient.
3. Failed claims do not reduce recorded allocation, but they block payout until more ARM is funded.

X Violated here: <https://prover.certora.com/output/727987/ded6c78a85bc4004ab692351e12a1766/>

✓ Passed After Fix: <https://prover.certora.com/output/727987/8294a3a9ee7c48cd9196bc7543fd3510/>

Setup and Execution

The Certora Prover can be run remotely using Certora's cloud infrastructure or locally from a source build. For this

engagement, the report metadata records Certora Prover version **8.8.1**. The commands below assume you are inside the repository's **contracts/** directory, where **certora/** is a direct child.

Common Setup (Ubuntu 24.04)

The first four steps are shared by remote and local execution.

Step 1 - Install Java (JDK 21)

```
sudo apt update
sudo apt install -y openjdk-21-jdk
java -version
```

Step 2 - Install pipx

pipx installs Python CLI tools in isolated environments, which keeps Certora dependencies separate from the repo's Python packages. See the [pipx documentation](#) for platform-specific notes.

```
sudo apt install -y pipx
pipx ensurepath
exec "$SHELL"
```

Step 3 - Install the Certora CLI

```
pipx install certora-cli==8.8.1
certoraRun --version
```

Step 4 - Install the Solidity Compiler Used by Armada

Install the Solidity compiler version required by the Armada repository and expose any compiler aliases referenced by the .conf files. If a config expects a named binary such as solc8.xx, point that alias at the selected solc binary.

```
pipx install solc-select

# Inspect the project for the expected compiler version or alias.
grep -R "solc" -n certora/confs foundry.toml 2>/dev/null || true

# Replace <project_solc_version> with the version required by the repository.
solc-select install <project_solc_version>
solc-select use <project_solc_version>
solc --version

# Optional: expose an alias if a Certora config references one.
mkdir -p "$HOME/.local/bin"
ln -sf "$(command -v solc)" "$HOME/.local/bin/solc<project_solc_version>"
export PATH="$HOME/.local/bin:$PATH"
```

Remote Execution

A Certora key is required for cloud runs. Certora provides keys through its [Discord](#) and website.

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
source ~/.bashrc
```

If a local Prover is installed, it takes priority by default. To force cloud execution for an Armada config, add --server production.

```
certoraRun certora/confs/Governance/RevenueLock.conf --server production
certoraRun certora/confs/Crowdfund/CFAccounting.conf --server production
```

Local Execution

For a local build matching this engagement, follow the [CertoraProver v8.8.1 repository](#). Once installed, the local Prover is

used before the cloud backend unless --server production is passed.

Install Local-Prover Prerequisites

```
# JDK 21
sudo apt install -y openjdk-21-jdk

# SMT solvers: z3 and cvc5 are required; place their binaries on PATH.
# z3:   https://github.com/Z3Prover/z3/releases
# cvc5: https://github.com/cvc5/cvc5/releases

# LLVM tools
sudo apt install -y llvm

# Rust 1.81.0+
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
cargo install rustfilt

# Optional, for visual reports
sudo apt install -y graphviz
```

Build CertoraProver v8.8.1

```
export CERTORA="$HOME/CertoraProver/target/installed/"
mkdir -p "$CERTORA"
export PATH="$CERTORA:$PATH"

git clone --recurse-submodules https://github.com/Certora/CertoraProver.git
cd CertoraProver
git checkout tags/8.8.1
./gradlew assemble
```

Verify the Local Installation

```
certoraRun.py -h
cd Public/TestEVM/Counter
certoraRun counter.conf
```

Running Armada Verification

Run the commands below from the repository's `contracts/` directory. If running from the repository root, prefix the paths with `contracts/`.

Run the Full Armada Suite

```
chmod +x certora/scripts/certora_run_all.sh
bash certora/scripts/certora_run_all.sh
```

Run Representative Configurations by Scope

```
# Crowdfund
certoraRun certora/confs/Crowdfund/CFAccounting.conf
certoraRun certora/confs/Crowdfund/CFCommitment.conf
certoraRun certora/confs/Crowdfund/CFFinalization.conf

# Revenue / Wind-down
certoraRun certora/confs/Governance/RevenueCounter.conf
certoraRun certora/confs/Governance/RevenueLock.conf
certoraRun certora/confs/WindDown/ArmadaWindDown_extended.conf
certoraRun certora/confs/WindDown/ArmadaRedemption_extended.conf

# Treasury / governance support
certoraRun certora/confs/Governance/TreasuryGov.conf
```

```
certoraRun certora/confs/Governance/TreasuryGovValueTransfer.conf
certoraRun certora/confs/Governance/TreasurySteward.conf
certoraRun certora/confs/Governance/XRateLimit.conf

# ArmadaGovernor
certoraRun certora/confs/Governance/ArmadaGovernorLifecycle.conf
certoraRun certora/confs/Governance/ArmadaGovernorVotes.conf
certoraRun certora/confs/Governance/ArmadaGovernorVeto.conf
```

Run One Rule While Debugging

```
certoraRun certora/confs/Governance/RevenueLock.conf --rule VS_solveny
certoraRun certora/confs/Crowdfund/CFCommitment.conf --rule VT_commitExactDeltas
certoraRun certora/confs/Governance/ArmadaGovernorVeto.conf --rule ST_vetoSingleVetoRule
```

Compilation-Only Checks

Use compilation-only checks before long runs or after touching imports, summaries, harnesses, or config paths.

```
certoraRun certora/confs/Crowdfund/CFAccounting.conf --compilation_steps_only
certoraRun certora/confs/Governance/RevenueLock.conf --compilation_steps_only
certoraRun certora/confs/Governance/TreasuryGov.conf --compilation_steps_only
certoraRun certora/confs/WindDown/ArmadaRedemption_extended.conf --compilation_steps_only
certoraRun certora/confs/Governance/ArmadaGovernorLifecycle.conf --compilation_steps_only
```

Resources

To learn more about Certora formal verification, the resources below are useful for onboarding, writing stronger CVL properties, debugging prover behavior.

- [Updraft Assembly & Formal Verification Course](#) - Comprehensive video course covering assembly and formal verification from the ground up.
- [RareSkills Certora Book](#) - Structured tutorial covering CVL syntax, specification patterns, and common pitfalls.
- [Certora Tutorials](#) - Official Certora tutorials for guided Prover, CVL, config, invariant, and debugging workflows.
- [Certora Prover Documentation](#) - Official Prover reference covering installation, CLI options, CVL, approximations, reports, and dashboard usage.